

Project 4

Generated by Doxygen 1.10.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Buffer Class Reference	5
3.1.1 Detailed Description	6
3.1.2 Constructor & Destructor Documentation	6
3.1.2.1 Buffer()	6
3.1.2.2 ~Buffer()	7
3.1.3 Member Function Documentation	7
3.1.3.1 Pack()	7
3.1.3.2 Print()	8
3.1.3.3 Read()	9
3.1.3.4 Unpack()	9
3.1.3.5 Write()	10
3.1.4 Member Data Documentation	11
3.1.4.1 buffer	11
3.1.4.2 buffer_index	11
3.1.4.3 buffer_size	11
3.1.4.4 file	11
3.2 Buffer2 Class Reference	11
3.2.1 Detailed Description	13
3.2.2 Constructor & Destructor Documentation	14
3.2.2.1 Buffer2()	14
3.2.3 Member Function Documentation	14
3.2.3.1 CalculateRecordCount()	14
3.2.3.2 DRead()	15
3.2.3.3 Flush()	16
3.2.3.4 getLastRecordLength()	17
3.2.3.5 getRecordCount()	17
3.2.3.6 Pack()	17
3.2.3.7 Print()	18
3.2.3.8 Read()	19
3.2.3.9 setRecordCount()	20
3.2.3.10 Unpack()	21
3.2.3.11 UpdateFieldOrder()	22
3.2.3.12 Write()	23
3.2.4 Member Data Documentation	24
3.2.4.1 BUFFER_SIZE	24
3.2.4.2 fieldOrder	24

3.2.4.3 ibuffer	24
3.2.4.4 ibufferEndPos	24
3.2.4.5 ibufferIndex	24
3.2.4.6 ibufferStartPos	25
3.2.4.7 lastRecordLength	25
3.2.4.8 obuffer	25
3.2.4.9 obufferIndex	25
3.2.4.10 recordCount	25
3.3 HeaderBuffer Class Reference	25
3.3.1 Detailed Description	26
3.3.2 Member Function Documentation	26
3.3.2.1 getMaxHeaderSize()	26
3.3.2.2 ReadHeader()	26
3.3.2.3 WriteHeader()	28
3.3.3 Member Data Documentation	29
3.3.3.1 buffer	29
3.3.3.2 MAX_HEADER_SIZE	29
3.4 HeaderRecord Class Reference	29
3.4.1 Detailed Description	32
3.4.2 Constructor & Destructor Documentation	32
3.4.2.1 HeaderRecord()	32
3.4.3 Member Function Documentation	33
3.4.3.1 getFieldOrder()	33
3.4.3.2 getFieldsPerRecord()	33
3.4.3.3 getFieldType()	33
3.4.3.4 getFileStructureType()	34
3.4.3.5 getHeaderRecordSize()	34
3.4.3.6 getIndexFileName()	35
3.4.3.7 getIndexFileSchemaInfo()	35
3.4.3.8 getPrimaryKey()	35
3.4.3.9 getRecordCount()	36
3.4.3.10 getRecordSizeInteger()	36
3.4.3.11 getSizeFormatType()	36
3.4.3.12 getVersion()	36
3.4.3.13 Pack()	37
3.4.3.14 Print()	37
3.4.3.15 setField()	37
3.4.3.16 setFieldsPerRecord()	38
3.4.3.17 setFileStructureType()	38
3.4.3.18 setHeaderRecordSize()	38
3.4.3.19 setIndexFileName()	39
3.4.3.20 setIndexFileSchemaInfo()	39

3.4.3.21 setPrimaryKey()	40
3.4.3.22 setRecordCount()	40
3.4.3.23 setRecordSizeInteger()	40
3.4.3.24 setSizeFormatType()	40
3.4.3.25 setVersion()	41
3.4.3.26 Unpack()	41
3.4.4 Member Data Documentation	41
3.4.4.1 fields	41
3.4.4.2 fieldsPerRecord	41
3.4.4.3 fileStructureType	42
3.4.4.4 headerRecordSize	42
3.4.4.5 indexFileName	42
3.4.4.6 indexFileSchemaInfo	42
3.4.4.7 primaryKey	42
3.4.4.8 recordCount	42
3.4.4.9 recordSizeInteger	42
3.4.4.10 sizeFormatType	42
3.4.4.11 version	43
3.5 PrimaryKeyIndex Class Reference	43
3.5.1 Detailed Description	44
3.5.2 Constructor & Destructor Documentation	44
3.5.2.1 PrimaryKeyIndex()	44
3.5.2.2 ~PrimaryKeyIndex()	44
3.5.3 Member Function Documentation	44
3.5.3.1 createIndex()	44
3.5.3.2 readIndexFromFile()	46
3.5.3.3 searchIndex()	46
3.5.3.4 setHeaderRecord()	47
3.5.3.5 writeIndexToFile()	48
3.5.4 Member Data Documentation	49
3.5.4.1 headerOffset	49
3.5.4.2 indexFile	49
3.5.4.3 intIndex	49
3.6 Zipcode Class Reference	50
3.6.1 Detailed Description	51
3.6.2 Constructor & Destructor Documentation	52
3.6.2.1 Zipcode()	52
3.6.3 Member Function Documentation	52
3.6.3.1 getCounty()	52
3.6.3.2 getLat()	53
3.6.3.3 getLon()	53
3.6.3.4 getPlaceName()	54

3.6.3.5 getState()	54
3.6.3.6 getZipcode()	55
3.6.3.7 Pack()	55
3.6.3.8 setCounty()	55
3.6.3.9 setLat()	56
3.6.3.10 setLon()	56
3.6.3.11 setPlaceName()	56
3.6.3.12 setState()	57
3.6.3.13 setZipcode()	57
3.6.3.14 Unpack()	57
3.6.4 Member Data Documentation	58
3.6.4.1 county	58
3.6.4.2 lat	58
3.6.4.3 lon	58
3.6.4.4 placeName	58
3.6.4.5 state	58
3.6.4.6 zipcode	59
3.7 Zipcode2 Class Reference	59
3.7.1 Detailed Description	61
3.7.2 Constructor & Destructor Documentation	61
3.7.2.1 Zipcode2()	61
3.7.3 Member Function Documentation	61
3.7.3.1 getCounty()	61
3.7.3.2 getLat()	61
3.7.3.3 getLon()	62
3.7.3.4 getPlaceName()	62
3.7.3.5 getState()	62
3.7.3.6 getZipcode()	63
3.7.3.7 isEmpty()	63
3.7.3.8 Pack()	63
3.7.3.9 Print()	64
3.7.3.10 setCounty()	64
3.7.3.11 setLat()	65
3.7.3.12 setLon()	65
3.7.3.13 setPlaceName()	66
3.7.3.14 setState()	66
3.7.3.15 setZipcode()	67
3.7.3.16 Unpack()	67
3.7.4 Member Data Documentation	68
3.7.4.1 county	68
3.7.4.2 lat	68
3.7.4.3 lon	68

3.7.4.4 placeName	68
3.7.4.5 state	68
3.7.4.6 zipcode	68
4 File Documentation	69
4.1 Project4Code/buffer.cpp File Reference	69
4.1.1 Detailed Description	69
4.2 Project4Code/buffer.h File Reference	70
4.3 buffer.h	70
4.4 Project4Code/buffer2.cpp File Reference	71
4.4.1 Detailed Description	71
4.5 Project4Code/buffer2.h File Reference	72
4.6 buffer2.h	72
4.7 Project4Code/fileConverter.cpp File Reference	73
4.7.1 Detailed Description	74
4.7.2 Function Documentation	74
4.7.2.1 main()	74
4.7.2.2 openFile()	75
4.7.2.3 readAndWriteRecords()	75
4.7.2.4 updateHeader()	76
4.7.2.5 writeDefaultHeader()	77
4.8 Project4Code/findZipcode.cpp File Reference	77
4.8.1 Detailed Description	78
4.8.2 Function Documentation	79
4.8.2.1 checkRecord()	79
4.8.2.2 InitializeProgram()	79
4.8.2.3 main()	80
4.8.2.4 processArguments()	82
4.9 Project4Code/headerBuffer.cpp File Reference	83
4.9.1 Detailed Description	84
4.10 Project4Code/headerBuffer.h File Reference	84
4.11 headerBuffer.h	85
4.12 Project4Code/headerRecord.cpp File Reference	85
4.12.1 Detailed Description	86
4.13 Project4Code/headerRecord.h File Reference	86
4.14 headerRecord.h	87
4.15 Project4Code/primaryKeyIndex.cpp File Reference	88
4.15.1 Detailed Description	88
4.16 Project4Code/primaryKeyIndex.h File Reference	88
4.17 primaryKeyIndex.h	89
4.18 Project4Code/proj3.cpp File Reference	90
4.18.1 Detailed Description	91

4.18.2 Function Documentation	91
4.18.2.1 main()	91
4.18.2.2 printData()	91
4.18.2.3 processData()	92
4.19 Project4Code/proj3Updated.cpp File Reference	93
4.19.1 Detailed Description	94
4.19.2 Function Documentation	94
4.19.2.1 main()	94
4.19.2.2 printData()	95
4.19.2.3 processData()	95
4.20 Project4Code/zipcode.cpp File Reference	96
4.20.1 Detailed Description	97
4.21 Project4Code/zipcode.h File Reference	97
4.21.1 Detailed Description	98
4.22 zipcode.h	98
4.23 Project4Code/zipcode2.cpp File Reference	99
4.23.1 Detailed Description	99
4.24 Project4Code/zipcode2.h File Reference	99
4.24.1 Detailed Description	100
4.25 zipcode2.h	100

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Buffer	A class to handle reading and writing of Zipcode objects to a file	5
Buffer2	A class to handle reading and writing of Zipcode2 objects to a file	11
HeaderBuffer	A class for managing the header of a ZRC file and associated IDX	25
HeaderRecord	A class to contain and process header information for a ZRC and IDX file	29
PrimaryKeyIndex	An index based on the primary key of the zipcode records	43
Zipcode	A class for handling zipcode records with associated geographic information	50
Zipcode2	A class for handling zipcode records with associated geographic information	59

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Project4Code/ buffer.cpp	69
This file contains the implementation of the Buffer class. It is used for project 3	
Project4Code/ buffer.h	70
This file contains the declaration of the Buffer class. It is used for project 3	
Project4Code/ buffer2.cpp	71
This file contains the implementation of the Buffer2 class. It is used for project 4	
Project4Code/ buffer2.h	72
This file contains the declaration of the Buffer2 class. It is used for project 4	
Project4Code/ fileConverter.cpp	73
A program to convert a CSV file into a custom ZCR (Zipcode Record) file	
Project4Code/ findZipcode.cpp	77
A program to find and display zip codes from a ZCR file	
Project4Code/ headerBuffer.cpp	83
This file contains the implementation of the HeaderBuffer class	
Project4Code/ headerBuffer.h	84
This file contains the declaration of the HeaderBuffer class	
Project4Code/ headerRecord.cpp	85
This file contains the implementation of the HeaderRecord class	
Project4Code/ headerRecord.h	86
This file contains the declaration of the HeaderRecord class	
Project4Code/ primaryKeyIndex.cpp	88
This file contains the implementation of the PrimaryKeyIndex class	
Project4Code/ primaryKeyIndex.h	88
This file contains the declaration of the PrimaryKeyIndex class	
Project4Code/ proj3.cpp	90
A program to analyze geographical data from a CSV file and display extreme zip codes for each state	
Project4Code/ proj3Updated.cpp	93
A program to analyze geographical data from a ZCR file and display extreme zip codes for each state	
Project4Code/ zipcode.cpp	96
This file contains the implementation of the Zipcode class. It is used for project 3	
Project4Code/ zipcode.h	97
This file contains the declaration of the Zipcode class. It is used for project 4	
Project4Code/ zipcode2.cpp	99
This file contains the implementation of the Zipcode2 class. It is used for project 4	
Project4Code/ zipcode2.h	99
This file contains the declaration of the Zipcode2 class. It is used for project 4	

Chapter 3

Class Documentation

3.1 Buffer Class Reference

A class to handle reading and writing of [Zipcode](#) objects to a file.

```
#include <buffer.h>
```

Collaboration diagram for Buffer:

Buffer
- file - buffer - buffer_index - buffer_size
+ Buffer() + ~Buffer() + Write() + Read() + Pack() + Unpack() + Print()

Public Member Functions

- [Buffer](#) (std::string filename)
Construct a new [Buffer](#) object.
- [~Buffer](#) ()

- Destroy the [Buffer](#) object.
- void [Write](#) ([Zipcode](#) &zipcode)
Write a [Zipcode](#) object to the file.
- [Zipcode Read](#) ()
Read a [Zipcode](#) object from the file.
- std::string [Pack](#) ([Zipcode](#) &zipcode)
Pack a [Zipcode](#) object into a string.
- [Zipcode Unpack](#) (std::string str)
Unpack a [Zipcode](#) object from a string.
- void [Print](#) ([Zipcode](#) &zipcode)
Print a [Zipcode](#) object.

Private Attributes

- std::fstream [file](#)
The file to read/write.
- std::vector< std::string > [buffer](#)
The buffer to hold the data.
- size_t [buffer_index](#)
The current index in the buffer.
- size_t [buffer_size](#) = 2000
The size of the buffer.

3.1.1 Detailed Description

A class to handle reading and writing of [Zipcode](#) objects to a file.

< Include the [Zipcode](#) class Include the fstream library for file input/output operations < Include the string library for string manipulation Include the vector library for dynamic array operations

The [Buffer](#) class is responsible for reading and writing [Zipcode](#) objects to a file. It uses a buffer to store data read from the file, and provides methods to pack and unpack [Zipcode](#) objects into strings that can be written to or read from the file. The buffer size is configurable, and the class provides error handling for file operations.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Buffer()

```
Buffer::Buffer (
    std::string filename )
```

Construct a new [Buffer](#) object.

< Include the vector library for dynamic array operations

Parameters

<i>filename</i>	The name of the file to read/write.
-----------------	-------------------------------------

< Include the [Buffer](#) class < Include the iostream library for input/output operations

Construct a new [Buffer](#) object and open the file.

Parameters

<i>filename</i>	The name of the file to open.
-----------------	-------------------------------

3.1.2.2 ~Buffer()

```
Buffer::~~Buffer ( )
```

Destroy the [Buffer](#) object.

Destroy the [Buffer](#) object and close the file if it is open.

3.1.3 Member Function Documentation

3.1.3.1 Pack()

```
std::string Buffer::Pack (
    Zipcode & zipcode )
```

Pack a [Zipcode](#) object into a string.

Parameters

<i>zipcode</i>	The Zipcode object to pack.
----------------	---

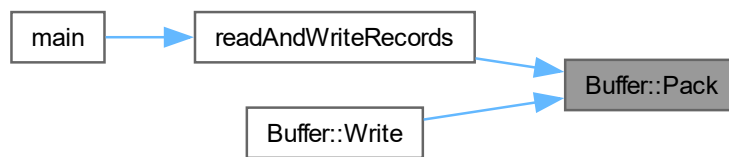
Returns

The packed string.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.2 Print()

```
void Buffer::Print (
    Zipcode & zipcode )
```

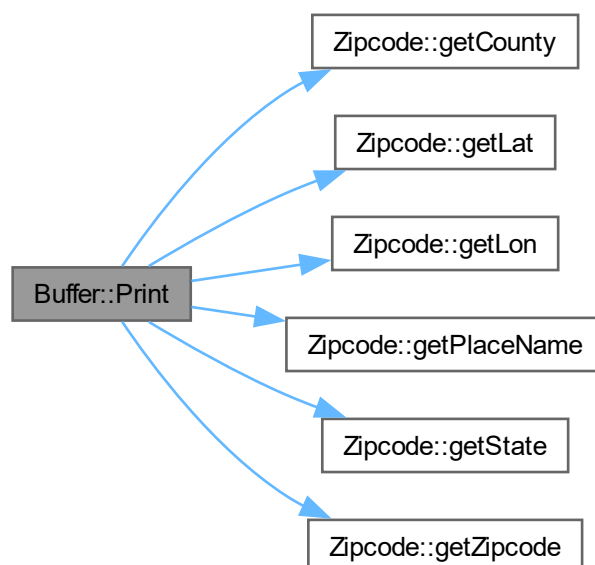
Print a [Zipcode](#) object.

Print the contents of a [Zipcode](#) object.

Parameters

<code>zipcode</code>	The Zipcode object to print.
----------------------	--

Here is the call graph for this function:



3.1.3.3 Read()

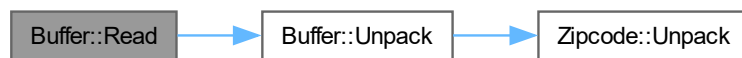
```
Zipcode Buffer::Read ( )
```

Read a [Zipcode](#) object from the file.

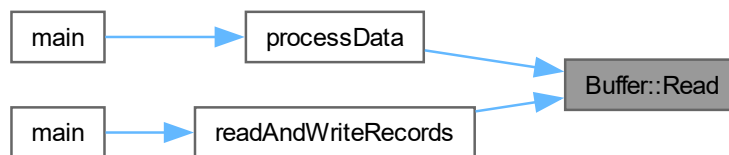
Returns

The [Zipcode](#) object read from the file.

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.3.4 Unpack()

```
Zipcode Buffer::Unpack (
    std::string str )
```

Unpack a [Zipcode](#) object from a string.

Unpack a string into a [Zipcode](#) object.

Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

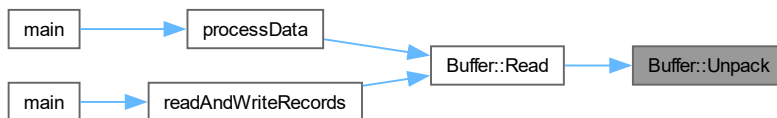
Returns

The unpacked [Zipcode](#) object.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.1.3.5 Write()**

```
void Buffer::Write (
    Zipcode & zipcode )
```

Write a [Zipcode](#) object to the file.

Parameters

<i>zipcode</i>	The Zipcode object to write.
----------------	--

Here is the call graph for this function:



3.1.4 Member Data Documentation

3.1.4.1 buffer

```
std::vector<std::string> Buffer::buffer [private]
```

The buffer to hold the data.

3.1.4.2 buffer_index

```
size_t Buffer::buffer_index [private]
```

The current index in the buffer.

3.1.4.3 buffer_size

```
size_t Buffer::buffer_size = 2000 [private]
```

The size of the buffer.

3.1.4.4 file

```
std::fstream Buffer::file [private]
```

The file to read/write.

The documentation for this class was generated from the following files:

- [Project4Code/buffer.h](#)
- [Project4Code/buffer.cpp](#)

3.2 Buffer2 Class Reference

A class to handle reading and writing of [Zipcode2](#) objects to a file.

```
#include <buffer2.h>
```

Collaboration diagram for Buffer2:

Buffer2
- obuffer - ibuffer - ibufferStartPos - ibufferEndPos - obufferIndex - ibufferIndex - BUFFER_SIZE - lastRecordLength - recordCount - fieldOrder
+ Buffer2() + setRecordCount() + getRecordCount() + getLastRecordLength() + Write() + Read() + DRead() + Pack() + Unpack() + Print() + Flush() + CalculateRecordCount() + UpdateFieldOrder()

Public Member Functions

- [Buffer2](#) ()
Construct a new [Buffer2](#) object.
- void [setRecordCount](#) (int count)
Set the Record Count.
- int [getRecordCount](#) () const
Get the Record Count.
- int [getLastRecordLength](#) () const
Get the Last Record Length.
- int [Write](#) ([Zipcode2](#) &zipcode, std::fstream &outFile)
Write a [Zipcode2](#) object to a file.

- [Zipcode2 Read](#) (std::fstream &inFile, int offset=-1)
Read a [Zipcode2](#) object from a file.
- [Zipcode2 DRead](#) (std::fstream &inFile, const int key, int offset)
Read a [Zipcode2](#) object from a file using a key.
- std::string [Pack](#) ([Zipcode2](#) &zipcode)
Pack a [Zipcode2](#) object into a string.
- [Zipcode2 Unpack](#) (std::string str)
Unpack a [Zipcode2](#) object from a string.
- void [Print](#) ([Zipcode2](#) &zipcode)
Print a [Zipcode2](#) object.
- void [Flush](#) (std::fstream &outFile)
Flush the output buffer to a file.
- int [CalculateRecordCount](#) (std::fstream &inFile)
Calculate the record count in a file.
- void [UpdateFieldOrder](#) (std::vector< std::string > newOrder)
Update the field order for packing/unpacking.

Private Attributes

- std::vector< std::string > [obuffer](#)
Output buffer.
- std::vector< std::string > [ibuffer](#)
Input buffer.
- std::streampos [ibufferStartPos](#)
Position in the file where the input buffer starts.
- std::streampos [ibufferEndPos](#)
Position in the file where the input buffer ends.
- size_t [obufferIndex](#)
Index into the output buffer.
- size_t [ibufferIndex](#)
Index into the input buffer.
- const size_t [BUFFER_SIZE](#) = 2000
Size of the buffer. Adjust this value as needed.
- int [lastRecordLength](#)
Length of the last record.
- int [recordCount](#)
Record count.
- std::vector< std::string > [fieldOrder](#)
Field order for packing/unpacking.

3.2.1 Detailed Description

A class to handle reading and writing of [Zipcode2](#) objects to a file.

A class to handle buffering operations for [Zipcode2](#) objects.

The [Buffer2](#) class is responsible for reading and writing [Zipcode2](#) objects to a file. It uses a buffer to store data read from the file, and provides methods to pack and unpack [Zipcode2](#) objects into strings that can be written to or read from the file. The buffer size is configurable, and the class provides error handling for file operations.

< Include the [Zipcode2](#) class Include the fstream library for file input/output operations < Include the string library for string manipulation Include the vector library for dynamic array operations

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Buffer2()

```
Buffer2::Buffer2 ( )
```

Construct a new [Buffer2](#) object.

< Include the sstream library for string stream operations

< Include the [Buffer2](#) class Include the iostream library for input/output operations Include the vector library for dynamic array operations

Construct a new [Buffer2](#) object.

Initializes the indices for the input and output buffers to 0.

3.2.3 Member Function Documentation

3.2.3.1 CalculateRecordCount()

```
int Buffer2::CalculateRecordCount (
    std::fstream & inFile )
```

Calculate the record count in a file.

Parameters

<i>inFile</i>	The input file stream.
---------------	------------------------

Returns

int The record count.

Calculates the number of records in the specified input file.

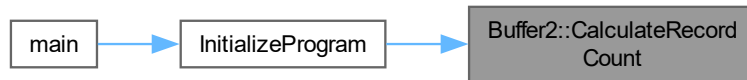
Parameters

<i>inFile</i>	The input file stream.
---------------	------------------------

Returns

int The record count.

Here is the caller graph for this function:

**3.2.3.2 DRead()**

```
Zipcode2 Buffer2::DRead (  
    std::fstream & inFile,  
    const int key,  
    int offset )
```

Read a [Zipcode2](#) object from a file using a key.

Parameters

<i>inFile</i>	The input file stream.
<i>key</i>	The key to use for reading.
<i>offset</i>	The offset in the file to start reading from.

Returns

[Zipcode2](#) The [Zipcode2](#) object read from the file.

Reads a [Zipcode2](#) object from the specified input file using a key. If the key does not match the zipcode read from the file, it returns an empty [Zipcode2](#) object.

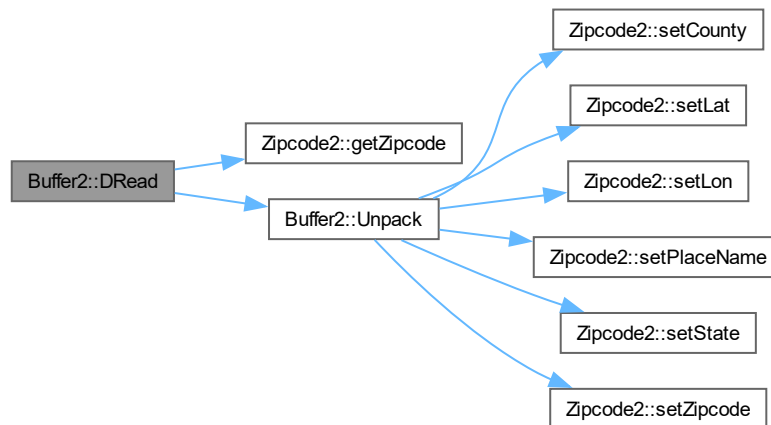
Parameters

<i>inFile</i>	The input file stream.
<i>key</i>	The key to use for reading.
<i>offset</i>	The offset in the file to start reading from.

Returns

[Zipcode2](#) The [Zipcode2](#) object read from the file. Returns an empty [Zipcode2](#) object if the file is not open, an invalid argument is encountered, or the key does not match.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.2.3.3 Flush()**

```
void Buffer2::Flush (
    std::fstream & outFile )
```

Flush the output buffer to a file.

Parameters

<code>outFile</code>	The output file stream.
----------------------	-------------------------

Writes all remaining records in the output buffer to the specified output file and then clears the output buffer.

Parameters

<code>outFile</code>	The output file stream.
----------------------	-------------------------

3.2.3.4 getLastRecordLength()

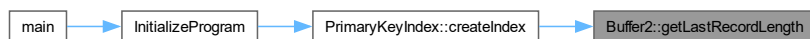
```
int Buffer2::getLastRecordLength ( ) const [inline]
```

Get the Last Record Length.

Returns

int The length of the last record.

Here is the caller graph for this function:



3.2.3.5 getRecordCount()

```
int Buffer2::getRecordCount ( ) const
```

Get the Record Count.

Returns

int The current record count.

3.2.3.6 Pack()

```
std::string Buffer2::Pack (
    Zipcode2 & zipcode )
```

Pack a [Zipcode2](#) object into a string.

Parameters

<i>zipcode</i>	The Zipcode2 object to pack.
----------------	--

Returns

std::string The packed [Zipcode2](#) object as a string.

Packs the given [Zipcode2](#) object into a string using the field order.

Parameters

<i>zipcode</i>	The Zipcode2 object to pack.
----------------	--

Returns

std::string The packed [Zipcode2](#) object as a string.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.2.3.7 Print()**

```
void Buffer2::Print (
    Zipcode2 & zipcode )
```

Print a [Zipcode2](#) object.

Parameters

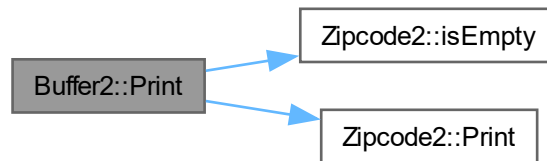
<i>zipcode</i>	The Zipcode2 object to print.
----------------	---

Prints the given [Zipcode2](#) object. If the [Zipcode2](#) object is empty, it prints a message indicating that the [Zipcode2](#) is empty.

Parameters

<i>zipcode</i>	The Zipcode2 object to print.
----------------	---

Here is the call graph for this function:



3.2.3.8 Read()

```

Zipcode2 Buffer2::Read (
    std::fstream & inFile,
    int offset = -1 )
  
```

Read a [Zipcode2](#) object from a file.

Parameters

<i>inFile</i>	The input file stream.
<i>offset</i>	The offset in the file to start reading from. Default is -1, which means read from the current position.

Returns

[Zipcode2](#) The [Zipcode2](#) object read from the file.

Reads a [Zipcode2](#) object from the specified input file. If the buffer is empty or has been fully read, it refills the buffer.

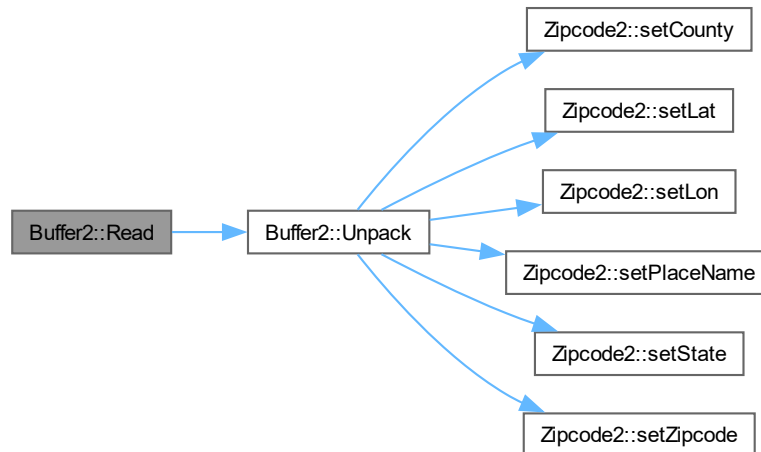
Parameters

<i>inFile</i>	The input file stream.
<i>offset</i>	The offset in the file to start reading from. Default is -1, which means read from the current position.

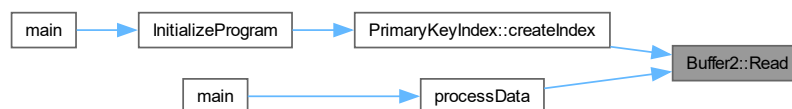
Returns

[Zipcode2](#) The [Zipcode2](#) object read from the file. Returns an empty [Zipcode2](#) object if the file is not open or an invalid argument is encountered.

Here is the call graph for this function:



Here is the caller graph for this function:

**3.2.3.9 setRecordCount()**

```
void Buffer2::setRecordCount (
    int count )
```

Set the Record Count.

Parameters

<i>count</i>	The new record count.
--------------	-----------------------

Sets the record count to the specified value.

Parameters

<i>count</i>	The new record count.
--------------	-----------------------

3.2.3.10 Unpack()

```
Zipcode2 Buffer2::Unpack (
    std::string str )
```

Unpack a [Zipcode2](#) object from a string.

Unpack a string into a [Zipcode2](#) object.

Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

Returns

[Zipcode2](#) The unpacked [Zipcode2](#) object.

Unpacks the given string into a [Zipcode2](#) object using the field order.

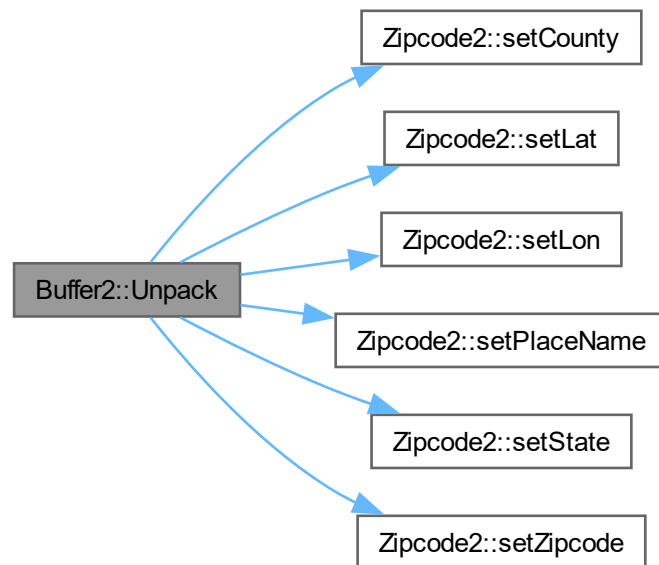
Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

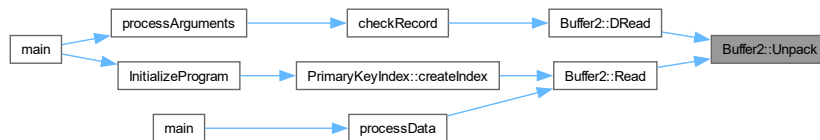
Returns

[Zipcode2](#) The unpacked [Zipcode2](#) object.

Here is the call graph for this function:



Here is the caller graph for this function:



3.2.3.11 UpdateFieldOrder()

```
void Buffer2::UpdateFieldOrder (
    std::vector< std::string > newOrder )
```

Update the field order for packing/unpacking.

Parameters

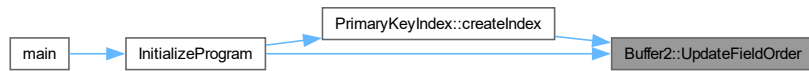
<i>newOrder</i>	The new field order.
-----------------	----------------------

Updates the field order for packing and unpacking [Zipcode2](#) objects.

Parameters

<i>newOrder</i>	The new field order.
-----------------	----------------------

Here is the caller graph for this function:



3.2.3.12 Write()

```
int Buffer2::Write (
    Zipcode2 & zipcode,
    std::fstream & outFile )
```

Write a [Zipcode2](#) object to a file.

Parameters

<i>zipcode</i>	The Zipcode2 object to write.
<i>outFile</i>	The output file stream.

Returns

int The number of characters written.

Writes the given [Zipcode2](#) object to the specified output file.

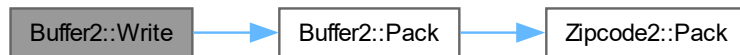
Parameters

<i>zipcode</i>	The Zipcode2 object to write.
<i>outFile</i>	The output file stream.

Returns

int The number of characters written. Returns -1 if the file is not open or the record format is invalid.

Here is the call graph for this function:



3.2.4 Member Data Documentation

3.2.4.1 BUFFER_SIZE

```
const size_t Buffer2::BUFFER_SIZE = 2000 [private]
```

Size of the buffer. Adjust this value as needed.

3.2.4.2 fieldOrder

```
std::vector<std::string> Buffer2::fieldOrder [private]
```

Field order for packing/unpacking.

3.2.4.3 ibuffer

```
std::vector<std::string> Buffer2::ibuffer [private]
```

Input buffer.

3.2.4.4 ibufferEndPos

```
std::streampos Buffer2::ibufferEndPos [private]
```

Position in the file where the input buffer ends.

3.2.4.5 ibufferIndex

```
size_t Buffer2::ibufferIndex [private]
```

Index into the input buffer.

3.2.4.6 ibufferStartPos

```
std::streampos Buffer2::ibufferStartPos [private]
```

Position in the file where the input buffer starts.

3.2.4.7 lastRecordLength

```
int Buffer2::lastRecordLength [private]
```

Length of the last record.

3.2.4.8 obuffer

```
std::vector<std::string> Buffer2::obuffer [private]
```

Output buffer.

3.2.4.9 obufferIndex

```
size_t Buffer2::obufferIndex [private]
```

Index into the output buffer.

3.2.4.10 recordCount

```
int Buffer2::recordCount [private]
```

Record count.

The documentation for this class was generated from the following files:

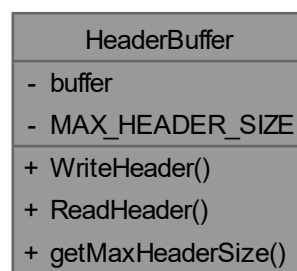
- [Project4Code/buffer2.h](#)
- [Project4Code/buffer2.cpp](#)

3.3 HeaderBuffer Class Reference

A class for managing the header of a ZRC file and associated IDX.

```
#include <headerBuffer.h>
```

Collaboration diagram for HeaderBuffer:



Public Member Functions

- int [WriteHeader](#) ([HeaderRecord](#) &hr, std::fstream &outFile)
Write the header to a file.
- bool [ReadHeader](#) ([HeaderRecord](#) &hr, std::fstream &inFile)
Read the header from a file.
- int [getMaxHeaderSize](#) () const
Get the maximum header size.

Private Attributes

- std::vector< std::string > [buffer](#)
The buffer to hold the contents of the header.
- const int [MAX_HEADER_SIZE](#) = 450
The maximum size of the header.

3.3.1 Detailed Description

A class for managing the header of a ZRC file and associated IDX.

< Include the vector library for dynamic array operations

The [HeaderBuffer](#) class is used to read and write the header of a data file. It holds the contents of the header in a buffer.

< Include the [HeaderRecord](#) class for handling the header record < Include the fstream library for file handling operations

A class to handle the header buffer.

3.3.2 Member Function Documentation

3.3.2.1 getMaxHeaderSize()

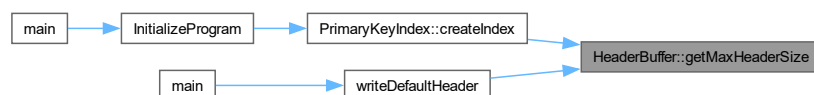
```
int HeaderBuffer::getMaxHeaderSize ( ) const [inline]
```

Get the maximum header size.

Returns

int The maximum header size.

Here is the caller graph for this function:



3.3.2.2 ReadHeader()

```
bool HeaderBuffer::ReadHeader (
    HeaderRecord & hr,
    std::fstream & file )
```

Read the header from a file.

Parameters

<i>hr</i>	The HeaderRecord object to read into.
<i>inFile</i>	The input file stream.

Returns

bool True if the header was read successfully, false otherwise.

Clears the buffer, reads the header from the file, unpacks the header into the [HeaderRecord](#) object, and checks for consistency.

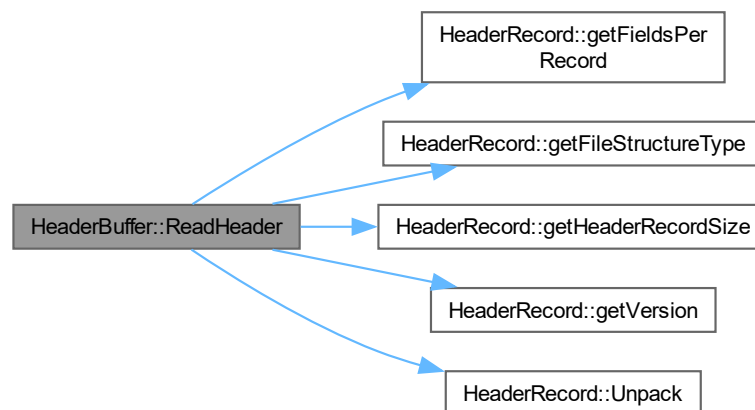
Parameters

<i>hr</i>	The HeaderRecord object to read into.
<i>file</i>	The input file stream.

Returns

bool True if the header was read successfully and is consistent, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



3.3.2.3 WriteHeader()

```
int HeaderBuffer::WriteHeader (
    HeaderRecord & hr,
    std::fstream & file )
```

Write the header to a file.

< Include the stringstream library for string manipulation

Parameters

<i>hr</i>	The HeaderRecord object to write.
<i>outFile</i>	The output file stream.

Returns

int The number of characters written.

< Include the [HeaderBuffer](#) class for handling the header buffer < Include the iostream library for input/output operations

Write the header to a file.

Converts the header record to a string, loads the header into the buffer, writes the buffer to the file, and returns the number of bytes written.

Parameters

<i>hr</i>	The HeaderRecord object to write.
<i>file</i>	The output file stream.

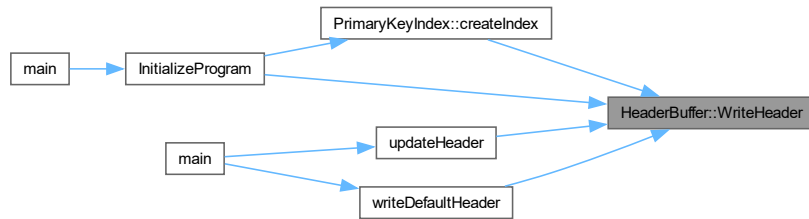
Returns

int The number of bytes written, or -1 if an error occurred.

Here is the call graph for this function:



Here is the caller graph for this function:



3.3.3 Member Data Documentation

3.3.3.1 buffer

```
std::vector<std::string> HeaderBuffer::buffer [private]
```

The buffer to hold the contents of the header.

3.3.3.2 MAX_HEADER_SIZE

```
const int HeaderBuffer::MAX_HEADER_SIZE = 450 [private]
```

The maximum size of the header.

The documentation for this class was generated from the following files:

- Project4Code/[headerBuffer.h](#)
- Project4Code/[headerBuffer.cpp](#)

3.4 HeaderRecord Class Reference

A class to contain and process header information for a ZRC and IDX file.

```
#include <headerRecord.h>
```

Collaboration diagram for HeaderRecord:

HeaderRecord
- fileStructureType - version - headerRecordSize - recordSizeInteger - sizeFormatType - indexFileName - indexFileSchemaInfo - recordCount - fieldsPerRecord - fields - primaryKey
+ HeaderRecord() + getFileStructureType() + getVersion() + getHeaderRecordSize() + getRecordSizeInteger() + getSizeFormatType() + getIndexFileName() + getIndexFileSchemaInfo() + getRecordCount() + getFieldsPerRecord() - and 17 more...

Public Member Functions

- [HeaderRecord](#) (int numFields=6)
Default constructor for [HeaderRecord](#).
- std::string [getFileStructureType](#) () const
Get the file structure type.
- std::string [getVersion](#) () const
Get the version of the file.
- int [getHeaderRecordSize](#) () const
Get the size of the header record.
- int [getRecordSizeInteger](#) () const
Get the size of the record as an integer.
- std::string [getSizeFormatType](#) () const
Get the format type of the size.

- `std::string getIndexFileName () const`
Get the name of the index file.
- `std::string getIndexFileSchemaInfo () const`
Get the schema information of the index file.
- `int getRecordCount () const`
Get the count of the records.
- `int getFieldsPerRecord () const`
Get the number of fields per record.
- `std::string getFieldType (const std::string &fieldName) const`
Get the type of a specific field.
- `std::string getPrimaryKey () const`
Get the primary key of the record.
- `void setFileStructureType (const std::string &value)`
Set the file structure type.
- `void setVersion (const std::string &value)`
Set the version of the file.
- `void setHeaderRecordSize (int value)`
Set the size of the header record.
- `void setRecordSizeInteger (int value)`
Set the size of the record as an integer.
- `void setSizeFormatType (const std::string &value)`
Set the format type of the size.
- `void setIndexFileName (const std::string &value)`
Set the name of the index file.
- `void setIndexFileSchemaInfo (const std::string &value)`
Set the schema information of the index file.
- `void setRecordCount (int value)`
Set the count of the records.
- `void setFieldsPerRecord (int value)`
Set the number of fields per record.
- `void setField (int index, const std::string &fieldName, const std::string &fieldType)`
Set a field in the record.
- `void setPrimaryKey (const std::string &value)`
Set the primary key of the record.
- `std::string Pack ()`
Pack the header record into a string.
- `void Unpack (std::string str)`
Unpack a string into a header record.
- `void Print ()`
Print the header record.
- `std::vector< std::string > getFieldOrder () const`
Get the order of the fields in the record.

Private Attributes

- `std::string fileStructureType`
The file structure type.
- `std::string version`
The version.
- `int headerRecordSize`

- The header record size.*
 - int [recordSizeInteger](#)
 - The record size as an integer.*
 - std::string [sizeFormatType](#)
 - The size format type.*
 - std::string [indexFileName](#)
 - The index file name.*
 - std::string [indexFileSchemaInfo](#)
 - The index file schema information.*
 - int [recordCount](#)
 - The record count.*
 - int [fieldsPerRecord](#)
 - The number of fields per record.*
 - std::vector< std::pair< std::string, std::string > > [fields](#)
 - The fields.*
 - std::string [primaryKey](#)
 - The primary key.*

3.4.1 Detailed Description

A class to contain and process header information for a ZRC and IDX file.

< Include the string library for string manipulation Include the vector library for dynamic array operations

The [HeaderRecord](#) class represents a header record in a file. It provides methods to pack and unpack the data into a string for storage, as well as getter and setter methods for each data field.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 HeaderRecord()

```
HeaderRecord::HeaderRecord (
    int numFields = 6 )
```

Default constructor for [HeaderRecord](#).

< Include the iomanip library for input/output manipulations

Parameters

<i>numFields</i>	The number of fields in the record. Default is 6.
------------------	---

< Include the header file for the [HeaderRecord](#) class < Include the sstream library for string stream operations < Include the iostream library for input/output stream operations

Constructor for [HeaderRecord](#).

Parameters

<i>numFields</i>	The number of fields in the record. Default is 6.
------------------	---

3.4.3 Member Function Documentation

3.4.3.1 getFieldOrder()

```
std::vector< std::string > HeaderRecord::getFieldOrder ( ) const
```

Get the order of the fields in the record.

Returns

A vector of strings representing the field order.

Here is the caller graph for this function:



3.4.3.2 getFieldsPerRecord()

```
int HeaderRecord::getFieldsPerRecord ( ) const [inline]
```

Get the number of fields per record.

Returns

The number of fields per record as an integer.

Here is the caller graph for this function:



3.4.3.3 getFieldType()

```
std::string HeaderRecord::getFieldType (
    const std::string & fieldName ) const
```

Get the type of a specific field.

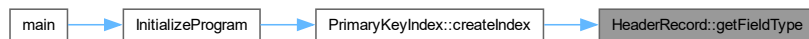
Parameters

<i>fieldName</i>	The name of the field.
------------------	------------------------

Returns

The type of the field as a string.

Here is the caller graph for this function:

**3.4.3.4 getFileStructureType()**

```
std::string HeaderRecord::getFileStructureType ( ) const [inline]
```

Get the file structure type.

Returns

The file structure type as a string.

Here is the caller graph for this function:

**3.4.3.5 getHeaderRecordSize()**

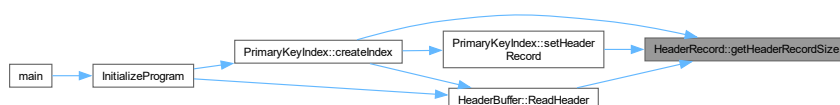
```
int HeaderRecord::getHeaderRecordSize ( ) const [inline]
```

Get the size of the header record.

Returns

The size of the header record as an integer.

Here is the caller graph for this function:



3.4.3.6 getIndexFileName()

```
std::string HeaderRecord::getIndexFileName ( ) const [inline]
```

Get the name of the index file.

Returns

The name of the index file as a string.

Here is the caller graph for this function:



3.4.3.7 getIndexFileSchemaInfo()

```
std::string HeaderRecord::getIndexFileSchemaInfo ( ) const [inline]
```

Get the schema information of the index file.

Returns

The schema information of the index file as a string.

Here is the caller graph for this function:



3.4.3.8 getPrimaryKey()

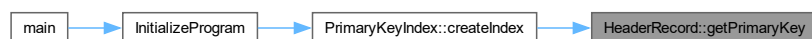
```
std::string HeaderRecord::getPrimaryKey ( ) const [inline]
```

Get the primary key of the record.

Returns

The primary key of the record as a string.

Here is the caller graph for this function:



3.4.3.9 getRecordCount()

```
int HeaderRecord::getRecordCount ( ) const [inline]
```

Get the count of the records.

Returns

The count of the records as an integer.

Here is the caller graph for this function:



3.4.3.10 getRecordSizeInteger()

```
int HeaderRecord::getRecordSizeInteger ( ) const [inline]
```

Get the size of the record as an integer.

Returns

The size of the record as an integer.

3.4.3.11 getSizeFormatType()

```
std::string HeaderRecord::getSizeFormatType ( ) const [inline]
```

Get the format type of the size.

Returns

The format type of the size as a string.

3.4.3.12 getVersion()

```
std::string HeaderRecord::getVersion ( ) const [inline]
```

Get the version of the file.

Returns

The version as a string.

Here is the caller graph for this function:



3.4.3.13 Pack()

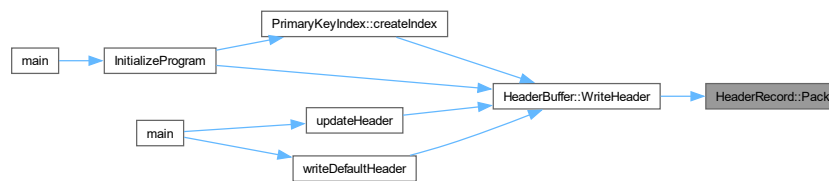
```
std::string HeaderRecord::Pack ( )
```

Pack the header record into a string.

Returns

The packed header record as a string.

Here is the caller graph for this function:



3.4.3.14 Print()

```
void HeaderRecord::Print ( )
```

Print the header record.

3.4.3.15 setField()

```
void HeaderRecord::setField (
    int index,
    const std::string & fieldName,
    const std::string & fieldType )
```

Set a field in the record.

Parameters

<i>index</i>	The index of the field.
<i>fieldName</i>	The name of the field.
<i>fieldType</i>	The type of the field.

Here is the caller graph for this function:



3.4.3.16 setFieldsPerRecord()

```
void HeaderRecord::setFieldsPerRecord (
    int value )
```

Set the number of fields per record.

Parameters

<i>value</i>	The number of fields per record as an integer.
--------------	--

3.4.3.17 setFileStructureType()

```
void HeaderRecord::setFileStructureType (
    const std::string & value )
```

Set the file structure type.

Parameters

<i>value</i>	The file structure type as a string.
--------------	--------------------------------------

Here is the caller graph for this function:



3.4.3.18 setHeaderRecordSize()

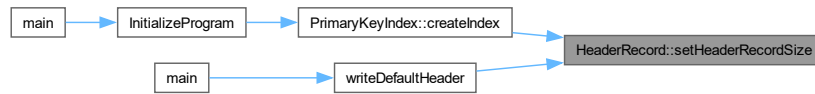
```
void HeaderRecord::setHeaderRecordSize (
    int value )
```

Set the size of the header record.

Parameters

<i>value</i>	The size of the header record as an integer.
--------------	--

Here is the caller graph for this function:



3.4.3.19 setIndexFileName()

```
void HeaderRecord::setIndexFileName (
    const std::string & value )
```

Set the name of the index file.

Parameters

<i>value</i>	The name of the index file as a string.
--------------	---

Here is the caller graph for this function:



3.4.3.20 setIndexFileSchemaInfo()

```
void HeaderRecord::setIndexFileSchemaInfo (
    const std::string & value )
```

Set the schema information of the index file.

Parameters

<i>value</i>	The schema information of the index file as a string.
--------------	---

Here is the caller graph for this function:



3.4.3.21 setPrimaryKey()

```
void HeaderRecord::setPrimaryKey (
    const std::string & value )
```

Set the primary key of the record.

Parameters

<i>value</i>	The primary key of the record as a string.
--------------	--

3.4.3.22 setRecordCount()

```
void HeaderRecord::setRecordCount (
    int value )
```

Set the count of the records.

Parameters

<i>value</i>	The count of the records as an integer.
--------------	---

Here is the caller graph for this function:



3.4.3.23 setRecordSizeInteger()

```
void HeaderRecord::setRecordSizeInteger (
    int value )
```

Set the size of the record as an integer.

Parameters

<i>value</i>	The size of the record as an integer.
--------------	---------------------------------------

3.4.3.24 setSizeFormatType()

```
void HeaderRecord::setSizeFormatType (
    const std::string & value )
```


Set the format type of the size.

Parameters

<i>value</i>	The format type of the size as a string.
--------------	--

3.4.3.25 setVersion()

```
void HeaderRecord::setVersion (
    const std::string & value )
```

Set the version of the file.

Parameters

<i>value</i>	The version as a string.
--------------	--------------------------

3.4.3.26 Unpack()

```
void HeaderRecord::Unpack (
    std::string str )
```

Unpack a string into a header record.

Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

Here is the caller graph for this function:



3.4.4 Member Data Documentation

3.4.4.1 fields

```
std::vector<std::pair<std::string, std::string> > HeaderRecord::fields [private]
```

The fields.

3.4.4.2 fieldsPerRecord

```
int HeaderRecord::fieldsPerRecord [private]
```

The number of fields per record.

3.4.4.3 fileStructureType

```
std::string HeaderRecord::fileStructureType [private]
```

The file structure type.

3.4.4.4 headerRecordSize

```
int HeaderRecord::headerRecordSize [private]
```

The header record size.

3.4.4.5 indexFileName

```
std::string HeaderRecord::indexFileName [private]
```

The index file name.

3.4.4.6 indexFileSchemaInfo

```
std::string HeaderRecord::indexFileSchemaInfo [private]
```

The index file schema information.

3.4.4.7 primaryKey

```
std::string HeaderRecord::primaryKey [private]
```

The primary key.

3.4.4.8 recordCount

```
int HeaderRecord::recordCount [private]
```

The record count.

3.4.4.9 recordSizeInteger

```
int HeaderRecord::recordSizeInteger [private]
```

The record size as an integer.

3.4.4.10 sizeFormatType

```
std::string HeaderRecord::sizeFormatType [private]
```

The size format type.

3.4.4.11 version

```
std::string HeaderRecord::version [private]
```

The version.

The documentation for this class was generated from the following files:

- Project4Code/[headerRecord.h](#)
- Project4Code/[headerRecord.cpp](#)

3.5 PrimaryKeyIndex Class Reference

An index based on the primary key of the zipcode records.

```
#include <primaryKeyIndex.h>
```

Collaboration diagram for PrimaryKeyIndex:

PrimaryKeyIndex
- indexFile
- intIndex
- headerOffset
+ PrimaryKeyIndex()
+ ~PrimaryKeyIndex()
+ createIndex()
+ writeIndexToFile()
+ readIndexFromFile()
+ searchIndex()
+ setHeaderRecord()

Public Member Functions

- [PrimaryKeyIndex](#) ()
Construct a new [PrimaryKeyIndex](#) object.
- [~PrimaryKeyIndex](#) ()
Destroy the [PrimaryKeyIndex](#) object.
- bool [createIndex](#) (std::fstream &inFile)
Create an index from a file.
- void [writeIndexToFile](#) (std::fstream &outFile)
Write the index to a file.
- void [readIndexFromFile](#) (std::fstream &inFile)
Read the index from a file.
- int [searchIndex](#) (int zipcode)
Search the index for a zipcode.
- void [setHeaderRecord](#) ([HeaderRecord](#) &dataHR, [HeaderRecord](#) &indexHR)
Set the [HeaderRecord](#) objects for the data and index files.

Private Attributes

- `std::fstream` [indexFile](#)
The index file stream.
- `std::map< int, int >` [intIndex](#)
The primary key index.
- `int` [headerOffset](#) = 450
The offset of the header in the file. This is used to skip the header when reading the records.

3.5.1 Detailed Description

An index based on the primary key of the zipcode records.

< Include the map library for storing the primary key index

This class is defined in project 3.

< Include the [HeaderRecord](#) class for handling the header record < Include the fstream library for file handling operations

A class to handle the primary key index.

3.5.2 Constructor & Destructor Documentation

3.5.2.1 PrimaryKeyIndex()

```
PrimaryKeyIndex::PrimaryKeyIndex ( )
```

Construct a new [PrimaryKeyIndex](#) object.

< Include the stringstream library for string manipulation

< Include the [PrimaryKeyIndex](#) class for handling the primary key index < Include the [Zipcode2](#) class for handling zipcode data < Include the [HeaderBuffer](#) class for handling the header buffer Include the [HeaderRecord](#) class for handling the header record < Include the [Buffer2](#) class for handling the data buffer

Construct a new [PrimaryKeyIndex](#) object.

3.5.2.2 ~PrimaryKeyIndex()

```
PrimaryKeyIndex::~~PrimaryKeyIndex ( )
```

Destroy the [PrimaryKeyIndex](#) object.

Closes the index file if it is open.

3.5.3 Member Function Documentation

3.5.3.1 createIndex()

```
bool PrimaryKeyIndex::createIndex (
    std::fstream & inFile )
```

Create an index from a file.

Parameters

<i>inFile</i>	The input file stream.
---------------	------------------------

Returns

bool True if the index was created successfully, false otherwise.

Reads the header from the input file, updates the field order, opens the index file, generates keys, writes the header to the index file, and writes the index to the index file.

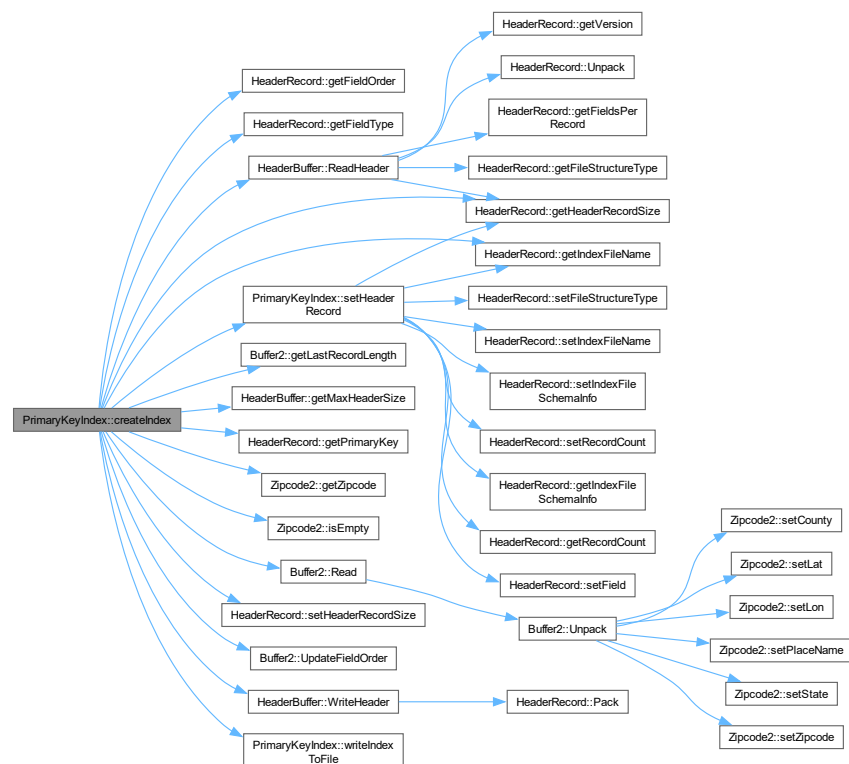
Parameters

<i>inFile</i>	The input file stream.
---------------	------------------------

Returns

bool True if the index was created successfully, false otherwise.

Here is the call graph for this function:



Here is the caller graph for this function:



3.5.3.2 readIndexFromFile()

```
void PrimaryKeyIndex::readIndexFromFile (
    std::fstream & inFile )
```

Read the index from a file.

Parameters

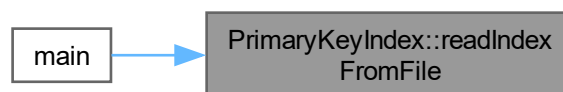
<i>inFile</i>	The input file stream.
---------------	------------------------

Reads each line from the input file and adds it to the index.

Parameters

<i>inFile</i>	The input file stream.
---------------	------------------------

Here is the caller graph for this function:



3.5.3.3 searchIndex()

```
int PrimaryKeyIndex::searchIndex (
    int zipcode )
```

Search the index for a zipcode.

Parameters

<i>zipcode</i>	The zipcode to search for.
----------------	----------------------------

Returns

int The index of the zipcode if found, -1 otherwise.

Searches the index for the given zipcode and returns the corresponding offset if found.

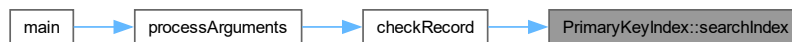
Parameters

<i>zipcode</i>	The zipcode to search for.
----------------	----------------------------

Returns

int The offset of the zipcode if found, -1 otherwise.

Here is the caller graph for this function:



3.5.3.4 setHeaderRecord()

```
void PrimaryKeyIndex::setHeaderRecord (
    HeaderRecord & dataHR,
    HeaderRecord & indexHR )
```

Set the [HeaderRecord](#) objects for the data and index files.

Parameters

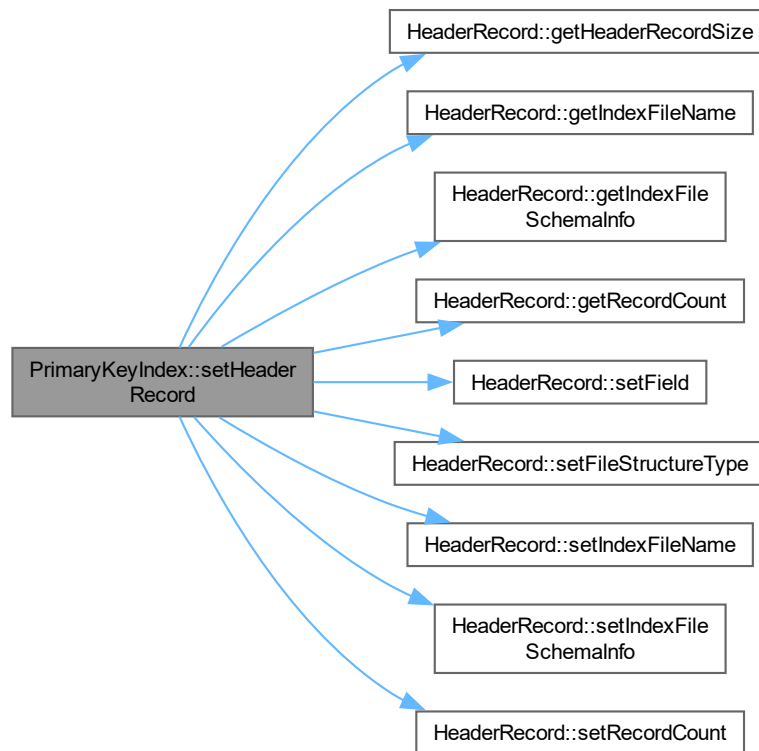
<i>dataHR</i>	The HeaderRecord object for the data file.
<i>indexHR</i>	The HeaderRecord object for the index file.

Updates the fields of the index [HeaderRecord](#) based on the data [HeaderRecord](#).

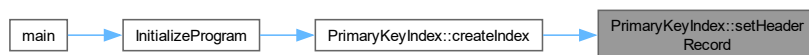
Parameters

<i>dataHR</i>	The HeaderRecord object for the data file.
<i>indexHR</i>	The HeaderRecord object for the index file.

Here is the call graph for this function:



Here is the caller graph for this function:



3.5.3.5 writeIndexToFile()

```
void PrimaryKeyIndex::writeIndexToFile (
    std::fstream & outFile )
```

Write the index to a file.

Parameters

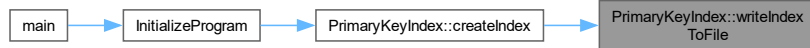
<code>outFile</code>	The output file stream.
----------------------	-------------------------

Writes each entry in the index to the output file.

Parameters

<i>outFile</i>	The output file stream.
----------------	-------------------------

Here is the caller graph for this function:



3.5.4 Member Data Documentation

3.5.4.1 headerOffset

```
int PrimaryKeyIndex::headerOffset = 450 [private]
```

The offset of the header in the file. This is used to skip the header when reading the records.

3.5.4.2 indexFile

```
std::fstream PrimaryKeyIndex::indexFile [private]
```

The index file stream.

3.5.4.3 intIndex

```
std::map<int, int> PrimaryKeyIndex::intIndex [private]
```

The primary key index.

The documentation for this class was generated from the following files:

- Project4Code/[primaryKeyIndex.h](#)
- Project4Code/[primaryKeyIndex.cpp](#)

3.6 Zipcode Class Reference

A class for handling zipcode records with associated geographic information.

```
#include <zipcode.h>
```

Collaboration diagram for Zipcode:

Zipcode
- zipcode - placeName - state - county - lat - lon
+ Zipcode() + Pack() + Unpack() + getZipcode() + getPlaceName() + getState() + getCounty() + getLat() + getLon() + setZipcode() + setPlaceName() + setState() + setCounty() + setLat() + setLon()

Public Member Functions

- [Zipcode](#) ()
Construct a new [Zipcode](#) object.
- std::string [Pack](#) ()
Pack the [Zipcode](#) object into a string for storage.
- void [Unpack](#) (std::string str)
Unpack a [Zipcode](#) object from a stored string.
- int [getZipcode](#) ()

- Get the Zipcode.*
- `std::string getPlaceName ()`
Get the Place Name.
- `std::string getState ()`
Get the State.
- `std::string getCounty ()`
Get the County.
- `double getLat ()`
Get the Latitude.
- `double getLon ()`
Get the Longitude.
- `void setZipcode (int zipcode)`
Set the Zipcode.
- `void setPlaceName (std::string placeName)`
Set the Place Name.
- `void setState (std::string state)`
Set the State.
- `void setCounty (std::string county)`
Set the County.
- `void setLat (double lat)`
Set the Latitude.
- `void setLon (double lon)`
Set the Longitude.

Private Attributes

- `int zipcode`
The zipcode.
- `std::string placeName`
The place name associated with the zipcode.
- `std::string state`
The state associated with the zipcode.
- `std::string county`
The county associated with the zipcode.
- `double lat`
The latitude of the zipcode.
- `double lon`
The longitude of the zipcode.

3.6.1 Detailed Description

A class for handling zipcode records with associated geographic information.

The `Zipcode` class provides methods to pack and unpack the data into a string for storage, as well as getter and setter methods for each data field. This class is used in project 3.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 Zipcode()

```
Zipcode::Zipcode ( )
```

Construct a new [Zipcode](#) object.

< Include the stringstream library for string manipulation

< Include the [Zipcode2](#) class

Construct a new [Zipcode](#) object.

Initializes all fields to their default values.

3.6.3 Member Function Documentation

3.6.3.1 getCounty()

```
std::string Zipcode::getCounty ( ) [inline]
```

Get the County.

Returns

std::string The county.

Here is the caller graph for this function:



3.6.3.2 getLat()

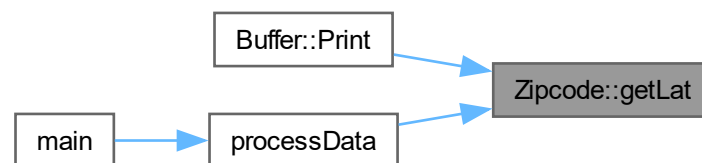
```
double Zipcode::getLat ( ) [inline]
```

Get the Latitude.

Returns

double The latitude.

Here is the caller graph for this function:



3.6.3.3 getLon()

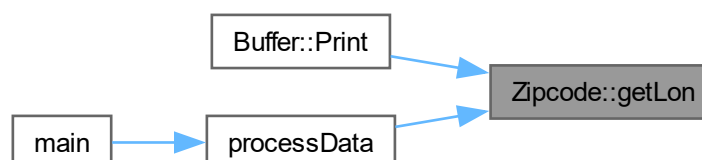
```
double Zipcode::getLon ( ) [inline]
```

Get the Longitude.

Returns

double The longitude.

Here is the caller graph for this function:



3.6.3.4 getPlaceName()

```
std::string Zipcode::getPlaceName ( ) [inline]
```

Get the Place Name.

Returns

std::string The place name.

Here is the caller graph for this function:



3.6.3.5 getState()

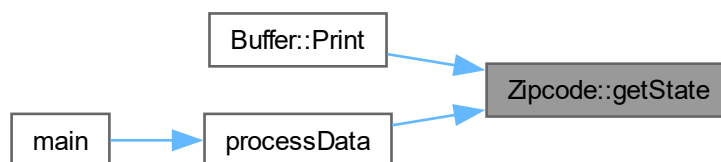
```
std::string Zipcode::getState ( ) [inline]
```

Get the State.

Returns

std::string The state.

Here is the caller graph for this function:



3.6.3.6 getZipcode()

```
int Zipcode::getZipcode ( ) [inline]
```

Get the [Zipcode](#).

Returns

int The zipcode.

Here is the caller graph for this function:



3.6.3.7 Pack()

```
std::string Zipcode::Pack ( )
```

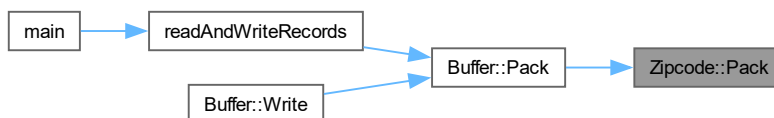
Pack the [Zipcode](#) object into a string for storage.

Packs the [Zipcode](#) object into a string.

Returns

The packed string.

Here is the caller graph for this function:



3.6.3.8 setCounty()

```
void Zipcode::setCounty (
    std::string newCounty )
```

Set the County.

Sets the county.

Parameters

<i>county</i>	The new county.
<i>newCounty</i>	The new county.

3.6.3.9 setLat()

```
void Zipcode::setLat (
    double newLat )
```

Set the Latitude.

Sets the latitude.

Parameters

<i>lat</i>	The new latitude.
<i>new_lat</i>	The new latitude.

3.6.3.10 setLon()

```
void Zipcode::setLon (
    double newLon )
```

Set the Longitude.

Sets the longitude.

Parameters

<i>lon</i>	The new longitude.
<i>newLon</i>	The new longitude.

3.6.3.11 setPlaceName()

```
void Zipcode::setPlaceName (
    std::string newPlaceName )
```

Set the Place Name.

Sets the place name.

Parameters

<i>placeName</i>	The new place name.
<i>newPlaceName</i>	The new place name.

3.6.3.12 setState()

```
void Zipcode::setState (
    std::string newState )
```

Set the State.

Sets the state.

Parameters

<i>state</i>	The new state.
<i>newState</i>	The new state.

3.6.3.13 setZipcode()

```
void Zipcode::setZipcode (
    int newZipcode )
```

Set the [Zipcode](#).

Sets the zipcode.

Parameters

<i>zipcode</i>	The new zipcode.
<i>newZipcode</i>	The new zipcode.

3.6.3.14 Unpack()

```
void Zipcode::Unpack (
    std::string str )
```

Unpack a [Zipcode](#) object from a stored string.

Unpacks a string into a [Zipcode](#) object.

Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

Here is the caller graph for this function:



3.6.4 Member Data Documentation

3.6.4.1 county

```
std::string Zipcode::county [private]
```

The county associated with the zipcode.

3.6.4.2 lat

```
double Zipcode::lat [private]
```

The latitude of the zipcode.

3.6.4.3 lon

```
double Zipcode::lon [private]
```

The longitude of the zipcode.

3.6.4.4 placeName

```
std::string Zipcode::placeName [private]
```

The place name associated with the zipcode.

3.6.4.5 state

```
std::string Zipcode::state [private]
```

The state associated with the zipcode.

3.6.4.6 zipcode

```
int Zipcode::zipcode [private]
```

The zipcode.

The documentation for this class was generated from the following files:

- Project4Code/[zipcode.h](#)
- Project4Code/[zipcode.cpp](#)

3.7 Zipcode2 Class Reference

A class for handling zipcode records with associated geographic information.

```
#include <zipcode2.h>
```

Collaboration diagram for Zipcode2:

Zipcode2
- zipcode - placeName - state - county - lat - lon
+ Zipcode2() + getZipcode() + getPlaceName() + getState() + getCounty() + getLat() + getLon() + setZipcode() + setPlaceName() + setState() - and 7 more...

Public Member Functions

- [Zipcode2](#) ()
Construct a new [Zipcode2](#) object.
- int [getZipcode](#) ()
Get the [Zipcode](#).
- std::string [getPlaceName](#) ()
Get the Place Name.
- std::string [getState](#) ()
Get the State.
- std::string [getCounty](#) ()
Get the County.
- double [getLat](#) ()
Get the Latitude.
- double [getLon](#) ()
Get the Longitude.
- void [setZipcode](#) (int [zipcode](#))
Set the [Zipcode](#).
- void [setPlaceName](#) (std::string [placeName](#))
Set the Place Name.
- void [setState](#) (std::string [state](#))
Set the State.
- void [setCounty](#) (std::string [county](#))
Set the County.
- void [setLat](#) (double [lat](#))
Set the Latitude.
- void [setLon](#) (double [lon](#))
Set the Longitude.
- std::string [Pack](#) (const std::vector< std::string > &fieldOrder)
Pack the [Zipcode2](#) object into a string.
- void [Unpack](#) (std::string str)
Unpack a string into a [Zipcode2](#) object.
- void [Print](#) ()
Print the [Zipcode2](#) object.
- bool [isEmpty](#) () const
Check if the [Zipcode2](#) object is empty.

Private Attributes

- int [zipcode](#)
The zipcode.
- std::string [placeName](#)
The place name.
- std::string [state](#)
The state.
- std::string [county](#)
The county.
- double [lat](#)
The latitude.
- double [lon](#)
The longitude.

3.7.1 Detailed Description

A class for handling zipcode records with associated geographic information.

< Include the string library for string manipulation Include the istream library for input/output operations Include the vector library for dynamic array operations

The [Zipcode2](#) class provides methods to pack and unpack the data into a string for storage, as well as getter and setter methods for each data field. This class is used in project 4.

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Zipcode2()

```
Zipcode2::Zipcode2 ( )
```

Construct a new [Zipcode2](#) object.

< Include the vector library for dynamic array operations

< Include the [Zipcode2](#) class < Include the stringstream library for string manipulation

Construct a new [Zipcode2](#) object.

Initializes all fields to their default values.

3.7.3 Member Function Documentation

3.7.3.1 getCounty()

```
std::string Zipcode2::getCounty ( ) [inline]
```

Get the County.

Returns

std::string The county.

3.7.3.2 getLat()

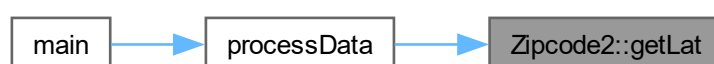
```
double Zipcode2::getLat ( ) [inline]
```

Get the Latitude.

Returns

double The latitude.

Here is the caller graph for this function:



3.7.3.3 getLon()

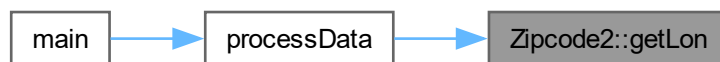
```
double Zipcode2::getLon ( ) [inline]
```

Get the Longitude.

Returns

double The longitude.

Here is the caller graph for this function:



3.7.3.4 getPlaceName()

```
std::string Zipcode2::getPlaceName ( ) [inline]
```

Get the Place Name.

Returns

std::string The place name.

3.7.3.5 getState()

```
std::string Zipcode2::getState ( ) [inline]
```

Get the State.

Returns

std::string The state.

Here is the caller graph for this function:



3.7.3.6 getZipcode()

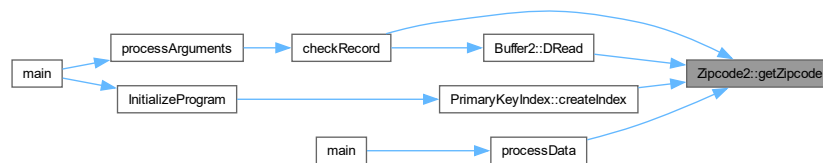
```
int Zipcode2::getZipcode ( ) [inline]
```

Get the [Zipcode](#).

Returns

int The zipcode.

Here is the caller graph for this function:



3.7.3.7 isEmpty()

```
bool Zipcode2::isEmpty ( ) const
```

Check if the [Zipcode2](#) object is empty.

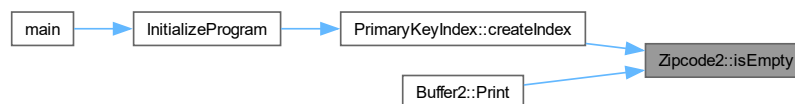
Check if all fields are in their default state.

Returns

bool True if the [Zipcode2](#) object is empty, false otherwise.

bool True if all fields are in their default state, false otherwise.

Here is the caller graph for this function:



3.7.3.8 Pack()

```
std::string Zipcode2::Pack (
    const std::vector< std::string > & fieldOrder )
```

Pack the [Zipcode2](#) object into a string.

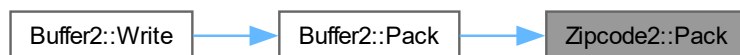
Parameters

<i>fieldOrder</i>	The order of the fields for packing.
-------------------	--------------------------------------

Returns

std::string The packed [Zipcode2](#) object as a string.

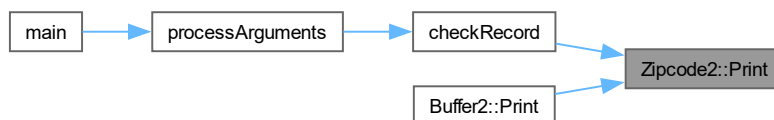
Here is the caller graph for this function:

**3.7.3.9 Print()**

```
void Zipcode2::Print ( )
```

Print the [Zipcode2](#) object.

Here is the caller graph for this function:

**3.7.3.10 setCounty()**

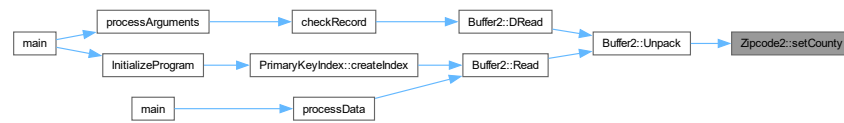
```
void Zipcode2::setCounty (
    std::string newCounty )
```

Set the County.

Parameters

<i>county</i>	The new county.
<i>newCounty</i>	The new county.

Here is the caller graph for this function:



3.7.3.11 setLat()

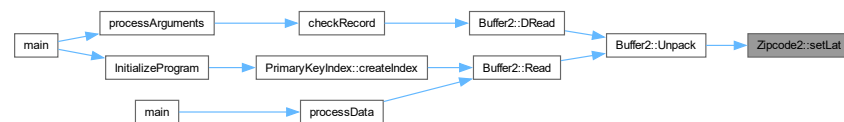
```
void Zipcode2::setLat (
    double newLat )
```

Set the Latitude.

Parameters

<i>lat</i>	The new latitude.
<i>newLat</i>	The new latitude.

Here is the caller graph for this function:



3.7.3.12 setLon()

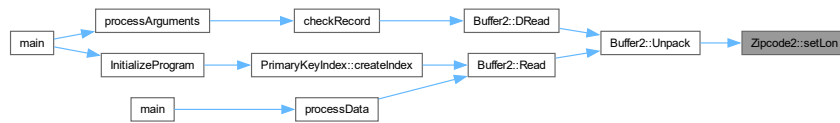
```
void Zipcode2::setLon (
    double newLon )
```

Set the Longitude.

Parameters

<i>lon</i>	The new longitude.
<i>newLon</i>	The new longitude.

Here is the caller graph for this function:



3.7.3.13 setPlaceName()

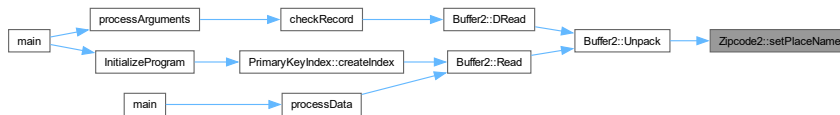
```
void Zipcode2::setPlaceName (
    std::string newPlaceName )
```

Set the Place Name.

Parameters

<i>placeName</i>	The new place name.
<i>newPlaceName</i>	The new place name.

Here is the caller graph for this function:



3.7.3.14 setState()

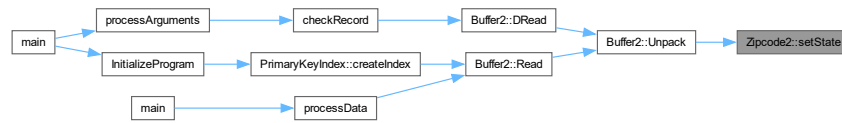
```
void Zipcode2::setState (
    std::string newState )
```

Set the State.

Parameters

<i>state</i>	The new state.
<i>newState</i>	The new state.

Here is the caller graph for this function:



3.7.3.15 setZipcode()

```
void Zipcode2::setZipcode (
    int newZipcode )
```

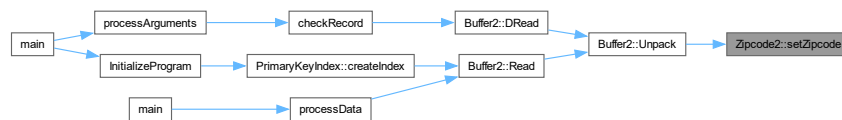
Set the [Zipcode](#).

Set the Zip Code.

Parameters

<i>zipcode</i>	The new zipcode.
<i>newZipcode</i>	The new zipcode.

Here is the caller graph for this function:



3.7.3.16 Unpack()

```
void Zipcode2::Unpack (
    std::string str )
```

Unpack a string into a [Zipcode2](#) object.

Parameters

<i>str</i>	The string to unpack.
------------	-----------------------

3.7.4 Member Data Documentation

3.7.4.1 county

```
std::string Zipcode2::county [private]
```

The county.

3.7.4.2 lat

```
double Zipcode2::lat [private]
```

The latitude.

3.7.4.3 lon

```
double Zipcode2::lon [private]
```

The longitude.

3.7.4.4 placeName

```
std::string Zipcode2::placeName [private]
```

The place name.

3.7.4.5 state

```
std::string Zipcode2::state [private]
```

The state.

3.7.4.6 zipcode

```
int Zipcode2::zipcode [private]
```

The zipcode.

The documentation for this class was generated from the following files:

- Project4Code/[zipcode2.h](#)
- Project4Code/[zipcode2.cpp](#)

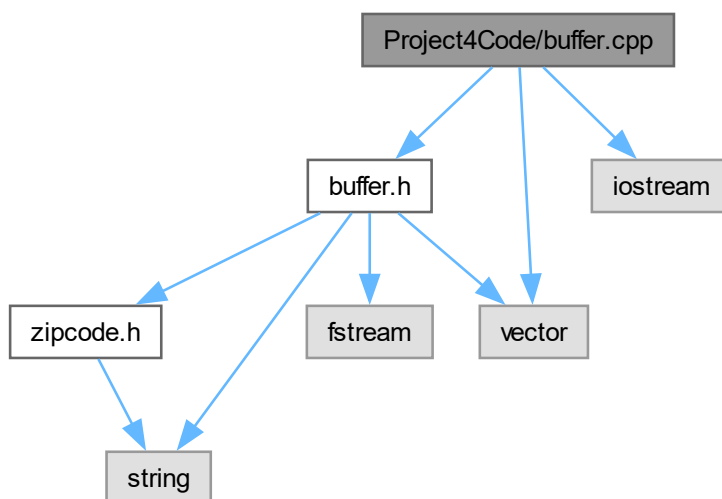
Chapter 4

File Documentation

4.1 Project4Code/buffer.cpp File Reference

This file contains the implementation of the [Buffer](#) class. It is used for project 3.

```
#include "buffer.h"  
#include <iostream>  
#include <vector>  
Include dependency graph for buffer.cpp:
```



4.1.1 Detailed Description

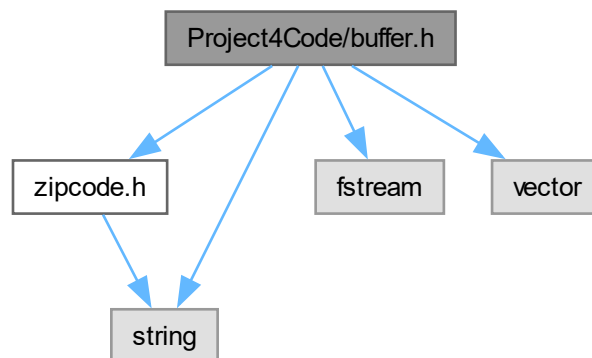
This file contains the implementation of the [Buffer](#) class. It is used for project 3.

4.2 Project4Code/buffer.h File Reference

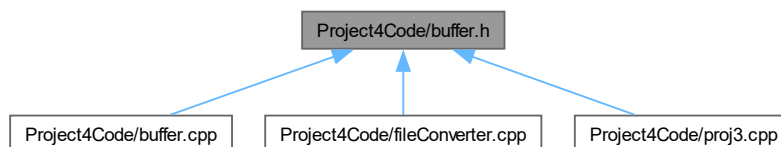
This file contains the declaration of the [Buffer](#) class. It is used for project 3.

```
#include "zipcode.h"
#include <fstream>
#include <string>
#include <vector>
```

Include dependency graph for buffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Buffer](#)
A class to handle reading and writing of [Zipcode](#) objects to a file.

4.2.1 Detailed Description

This file contains the declaration of the [Buffer](#) class. It is used for project 3.

4.3 buffer.h

[Go to the documentation of this file.](#)

```

00001
00012 #ifndef BUFFER_H
00013 #define BUFFER_H
00014
00015 #include "zipcode.h"
00016 #include <fstream>
00017 #include <string>
00018 #include <vector>
00019
00020 class Buffer {
00021 public:
00022     Buffer(std::string filename);
00027
00031     ~Buffer();
00032
00037     void Write(Zipcode& zipcode);
00038
00043     Zipcode Read();
00044
00050     std::string Pack(Zipcode& zipcode);
00051
00057     Zipcode Unpack(std::string str);
00058
00063     void Print(Zipcode& zipcode);
00064
00065 private:
00066     std::fstream file;
00067     std::vector<std::string> buffer;
00068     size_t buffer_index;
00069     size_t buffer_size = 2000;
00070 };
00071
00072 #endif

```

4.4 Project4Code/buffer2.cpp File Reference

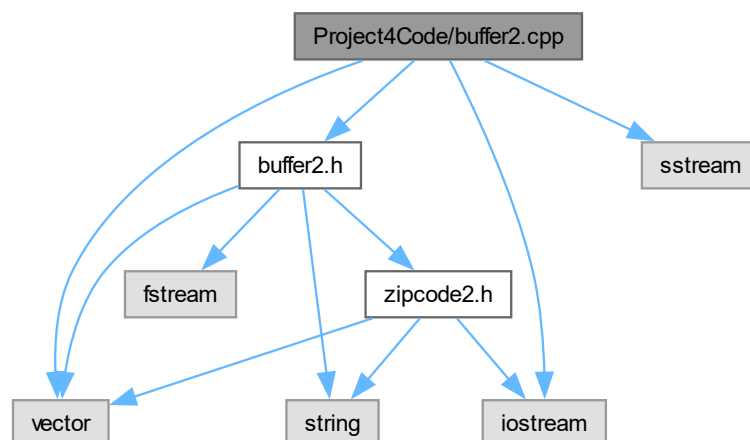
This file contains the implementation of the [Buffer2](#) class. It is used for project 4.

```

#include "buffer2.h"
#include <iostream>
#include <vector>
#include <sstream>

```

Include dependency graph for buffer2.cpp:



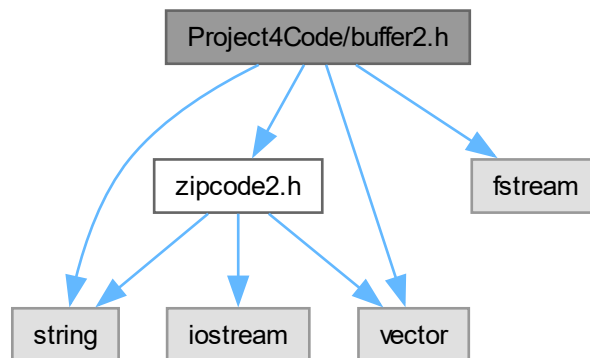
4.4.1 Detailed Description

This file contains the implementation of the [Buffer2](#) class. It is used for project 4.

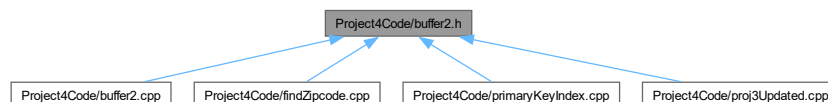
4.5 Project4Code/buffer2.h File Reference

This file contains the declaration of the [Buffer2](#) class. It is used for project 4.

```
#include "zipcode2.h"
#include <fstream>
#include <string>
#include <vector>
Include dependency graph for buffer2.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [Buffer2](#)
A class to handle reading and writing of [Zipcode2](#) objects to a file.

4.5.1 Detailed Description

This file contains the declaration of the [Buffer2](#) class. It is used for project 4.

4.6 buffer2.h

[Go to the documentation of this file.](#)

```

00001
00012 #ifndef BUFFER2_H
00013 #define BUFFER2_H
00014
00015 #include "zipcode2.h"
00016 #include <fstream>
00017 #include <string>
00018 #include <vector>
00019
00024 class Buffer2 {
00025 public:
00029     Buffer2();
00030
00035     void setRecordCount(int count);
00036
00041     int getRecordCount() const;
00042
00047     int getLastRecordLength() const { return lastRecordLength; };
00048
00055     int Write(Zipcode2& zipcode, std::fstream& outFile);
00056
00063     Zipcode2 Read(std::fstream& inFile, int offset = -1);
00064
00072     Zipcode2 DRead(std::fstream& inFile, const int key, int offset);
00073
00079     std::string Pack(Zipcode2& zipcode);
00080
00086     Zipcode2 Unpack(std::string str);
00087
00092     void Print(Zipcode2& zipcode);
00093
00098     void Flush(std::fstream& outFile);
00099
00105     int CalculateRecordCount(std::fstream& inFile);
00106
00111     void UpdateFieldOrder(std::vector<std::string> newOrder);
00112
00113 private:
00114     std::vector<std::string> obuffer;
00115     std::vector<std::string> ibuffer;
00116     std::streampos ibufferStartPos;
00117     std::streampos ibufferEndPos;
00118     size_t obufferIndex;
00119     size_t ibufferIndex;
00120     const size_t BUFFER_SIZE = 2000;
00121     int lastRecordLength;
00122     int recordCount;
00123     std::vector<std::string> fieldOrder;
00124 };
00125
00126 #endif

```

4.7 Project4Code/fileConverter.cpp File Reference

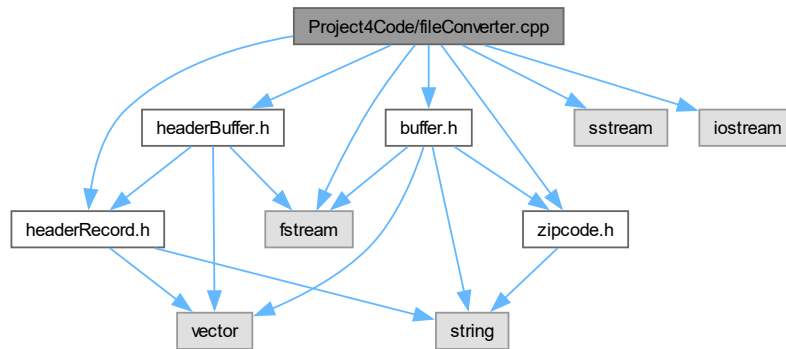
A program to convert a CSV file into a custom ZCR ([Zipcode](#) Record) file.

```

#include "headerBuffer.h"
#include "headerRecord.h"
#include "buffer.h"
#include "zipcode.h"
#include <fstream>
#include <sstream>
#include <iostream>

```

Include dependency graph for fileConverter.cpp:



Functions

- `std::fstream` [openFile](#) (`const std::string &filename`, `std::ios_base::openmode mode`)
< Include the `iostream` library for input/output operations
- `void` [writeDefaultHeader](#) (`std::fstream &zcrFile`)
Write a default header to the `.zcr` file.
- `int` [readAndWriteRecords](#) (`const std::string &csvFileName`, `std::fstream &zcrFile`)
Read the `.csv` and write records to `.zcr`.
- `void` [updateHeader](#) (`std::fstream &zcrFile`, `int recordCount`)
Update the header of the `.zcr` file.
- `int` [main](#) ()
The main function of the program.

4.7.1 Detailed Description

A program to convert a CSV file into a custom ZCR ([Zipcode](#) Record) file.

This program reads zipcode record data from a CSV file and converts it into a custom ZCR file that includes a header and length indicated records.

Author

Daniel Leone @starID [REDACTED]

Date

04/05/2024

4.7.2 Function Documentation

4.7.2.1 main()

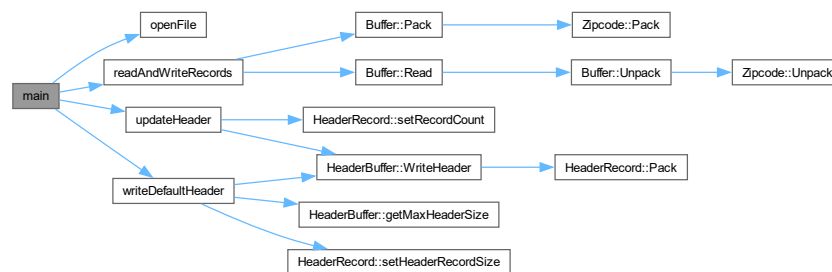
```
int main ( )
```

The main function of the program.

Returns

0 if the program executed successfully, and 1 otherwise.

Here is the call graph for this function:



4.7.2.2 openFile()

```
std::fstream openFile (
    const std::string & filename,
    std::ios_base::openmode mode )
```

< Include the `iostream` library for input/output operations

< Include the [HeaderBuffer](#) class for handling the header buffer Include the [HeaderRecord](#) class for handling the header record < Include the [Buffer](#) class for handling the data buffer < Include the [Zipcode](#) class for handling zipcode data < Include the `fstream` library for file handling operations Include the `sstream` library for string stream operations

Open a file with error checking.

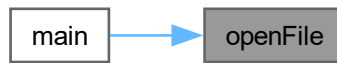
Parameters

<i>filename</i>	The name of the file to open.
<i>mode</i>	The mode in which to open the file.

Returns

The opened file stream.

Here is the caller graph for this function:



4.7.2.3 readAndWriteRecords()

```
int readAndWriteRecords (
    const std::string & csvFileName,
    std::fstream & zcrFile )
```

Read the .csv and write records to .zcr.

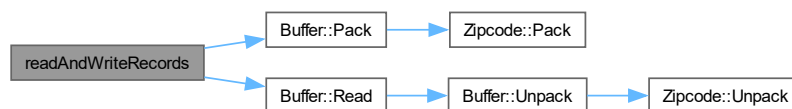
Parameters

<i>csvFileName</i>	The name of the .csv file to read from.
<i>zcrFile</i>	The .zcr file to write to.

Returns

The number of records written.

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.4 updateHeader()

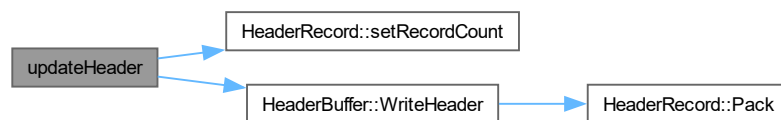
```
void updateHeader (
    std::fstream & zcrFile,
    int recordCount )
```

Update the header of the .zcr file.

Parameters

<i>zcrFile</i>	The .zcr file to update.
<i>recordCount</i>	The number of records to set in the header.

Here is the call graph for this function:



Here is the caller graph for this function:



4.7.2.5 writeDefaultHeader()

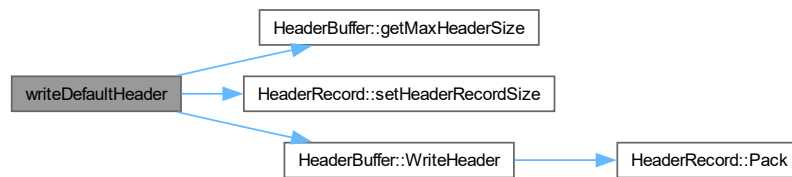
```
void writeDefaultHeader (
    std::fstream & zcrFile )
```

Write a default header to the .zcr file.

Parameters

<i>zcrFile</i>	The .zcr file to write to.
----------------	----------------------------

Here is the call graph for this function:



Here is the caller graph for this function:



4.8 Project4Code/findZipcode.cpp File Reference

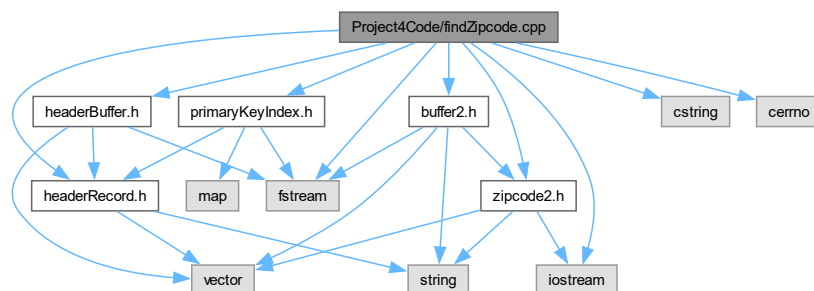
A program to find and display zip codes from a ZCR file.

```

#include "headerBuffer.h"
#include "headerRecord.h"
#include "buffer2.h"
#include "zipcode2.h"
#include "primaryKeyIndex.h"
#include <iostream>
#include <fstream>
#include <cstring>
#include <cerrno>

```

Include dependency graph for `findZipcode.cpp`:



Functions

- void `InitializeProgram` (std::fstream &dataFile, `HeaderBuffer` &hb, `Buffer2` &buffer, `PrimaryKeyIndex` &index, `HeaderRecord` &header)
< Include the cerrno library for error handling
- void `processArguments` (int argc, char *argv[], std::fstream &dataFile, `PrimaryKeyIndex` &index, `Buffer2` &buffer)
Process the command line arguments or user input to search for a record.
- int `main` (int argc, char *argv[])
The main function of the program.
- int `checkRecord` (std::fstream &dataFile, `PrimaryKeyIndex` &index, `Buffer2` &buffer, int zipcode)

4.8.1 Detailed Description

A program to find and display zip codes from a ZCR file.

This program reads zipcode record data from a custom ZCR file that includes a header and the records are length indicated. An index file (IDX) is generated and utilized to quickly locate specific records by zipcode. The record is located based on user input from the command line with the -Z flag. The record is displayed if found, or a message is displayed stating the record is not present. Multiple records can be accessed at one execution of the program.

Author

Daniel Leone @starID [REDACTED]

Date

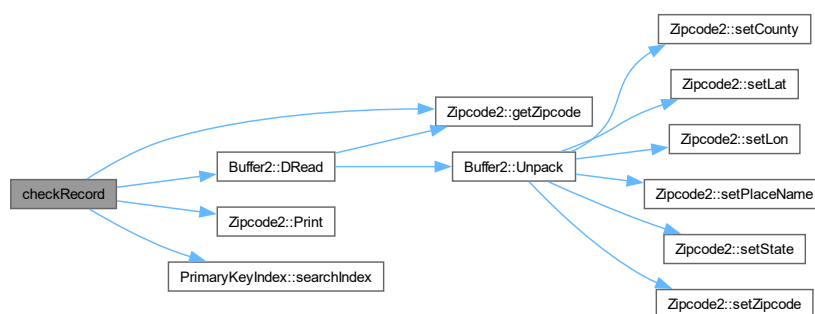
04/08/2024

4.8.2 Function Documentation

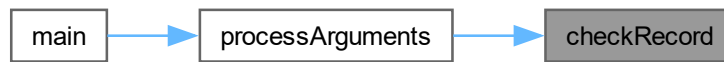
4.8.2.1 checkRecord()

```
int checkRecord (
    std::fstream & dataFile,
    PrimaryKeyIndex & index,
    Buffer2 & buffer,
    int zipcode )
```

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.2.2 InitializeProgram()

```

void InitializeProgram (
    std::fstream & dataFile,
    HeaderBuffer & hb,
    Buffer2 & buffer,
    PrimaryKeyIndex & index,
    HeaderRecord & header )
  
```

< Include the cerrno library for error handling

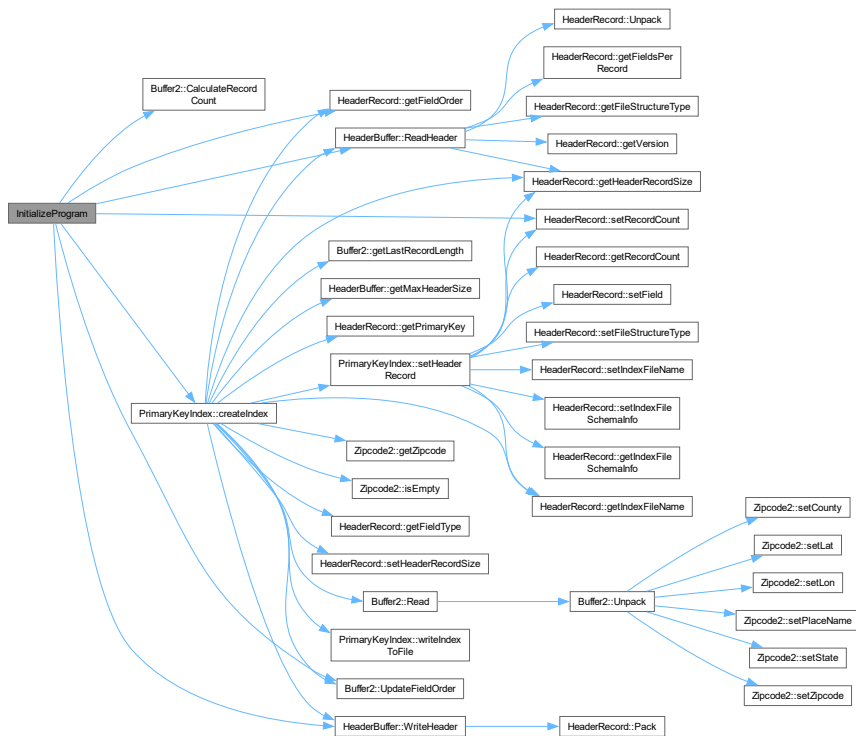
< Include the [HeaderBuffer](#) class for handling the header buffer Include the [HeaderRecord](#) class for handling the header record < Include the [Buffer2](#) class for handling the data buffer < Include the [Zipcode2](#) class for handling zipcode data < Include the [PrimaryKeyIndex](#) class for handling the primary key index < Include the iostream library for input/output operations < Include the fstream library for file handling operations Include the cstring library for string handling functions

Initialize the program by reading the header, updating the field order, calculating the record count, writing back the header, and creating the index file.

Parameters

<i>dataFile</i>	The data file to be read and written.
<i>hb</i>	The HeaderBuffer object.
<i>buffer</i>	The Buffer2 object.
<i>index</i>	The PrimaryKeyIndex object.
<i>header</i>	The HeaderRecord object.

Here is the call graph for this function:



Here is the caller graph for this function:



4.8.2.3 main()

```
int main (
    int argc,
    char * argv[] )
```

The main function of the program.

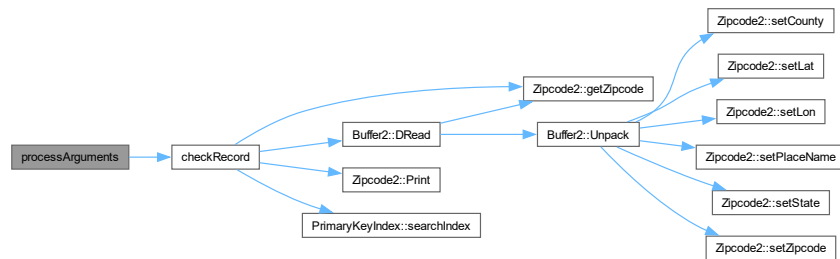
Parameters

<i>argc</i>	The number of command line arguments.
<i>argv</i>	The command line arguments.

Returns

void

Here is the call graph for this function:



Here is the caller graph for this function:



4.9 Project4Code/headerBuffer.cpp File Reference

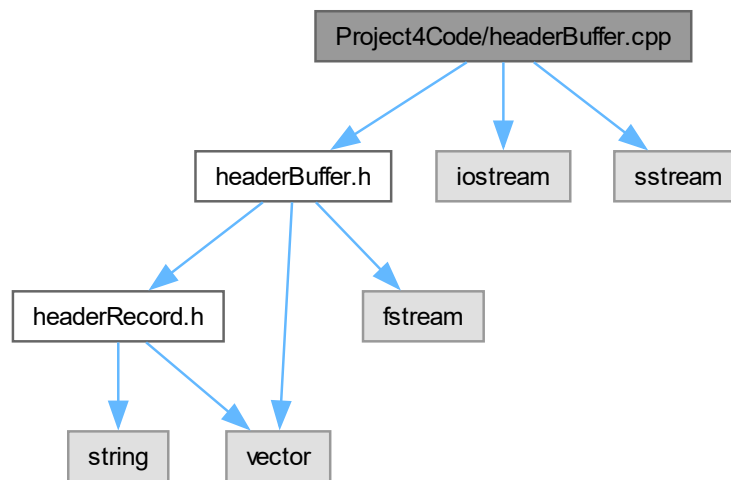
This file contains the implementation of the [HeaderBuffer](#) class.

```

#include "headerBuffer.h"
#include <iostream>
#include <sstream>

```

Include dependency graph for headerBuffer.cpp:



4.9.1 Detailed Description

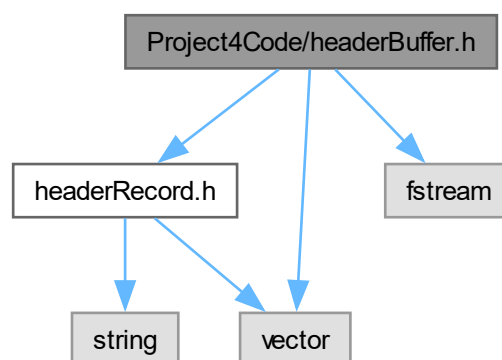
This file contains the implementation of the [HeaderBuffer](#) class.

4.10 Project4Code/headerBuffer.h File Reference

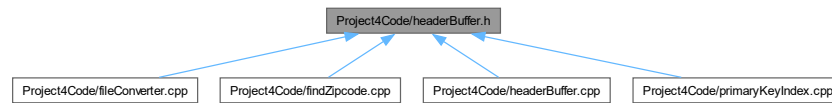
This file contains the declaration of the [HeaderBuffer](#) class.

```
#include "headerRecord.h"  
#include <fstream>  
#include <vector>
```

Include dependency graph for headerBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HeaderBuffer](#)

A class for managing the header of a ZRC file and associated IDX.

4.10.1 Detailed Description

This file contains the declaration of the [HeaderBuffer](#) class.

4.11 headerBuffer.h

[Go to the documentation of this file.](#)

```

00001
00009 #ifndef HEADER_BUFFER_H
00010 #define HEADER_BUFFER_H
00011
00012 #include "headerRecord.h"
00013 #include <fstream>
00014 #include <vector>
00015
00020 class HeaderBuffer {
00021 public:
00028     int WriteHeader(HeaderRecord& hr, std::fstream& outFile);
00029
00036     bool ReadHeader(HeaderRecord& hr, std::fstream& inFile);
00037
00042     int getMaxHeaderSize() const { return MAX_HEADER_SIZE; };
00043
00044 private:
00045     std::vector<std::string> buffer;
00046     const int MAX_HEADER_SIZE = 450;
00047 };
00048
00049 #endif // HEADER_BUFFER_H
  
```

4.12 Project4Code/headerRecord.cpp File Reference

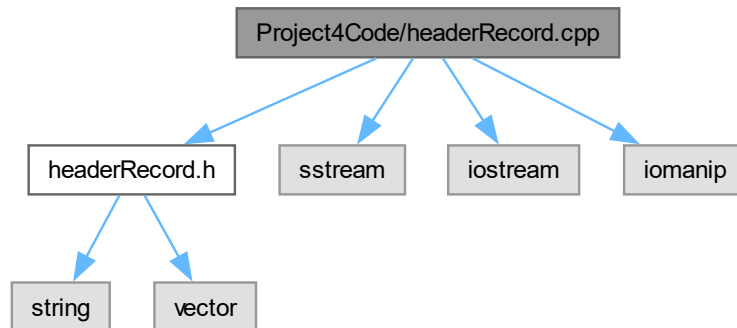
This file contains the implementation of the [HeaderRecord](#) class.

```

#include "headerRecord.h"
#include <sstream>
#include <iostream>
  
```

```
#include <iomanip>
```

Include dependency graph for headerRecord.cpp:



4.12.1 Detailed Description

This file contains the implementation of the [HeaderRecord](#) class.

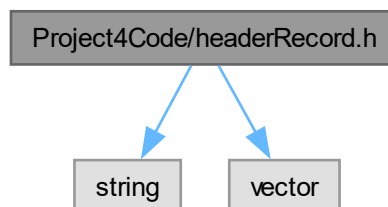
4.13 Project4Code/headerRecord.h File Reference

This file contains the declaration of the [HeaderRecord](#) class.

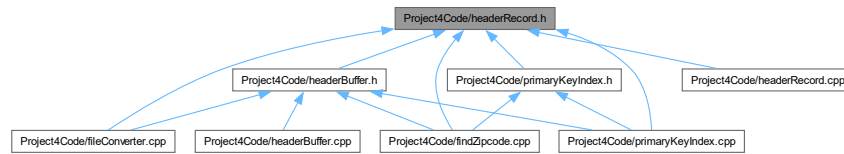
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for headerRecord.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HeaderRecord](#)

A class to contain and process header information for a ZRC and IDX file.

4.13.1 Detailed Description

This file contains the declaration of the [HeaderRecord](#) class.

4.14 headerRecord.h

[Go to the documentation of this file.](#)

```

00001
00009 #ifndef HEADER_RECORD_H
00010 #define HEADER_RECORD_H
00011
00012 #include <string>
00013 #include <vector>
00014
00015 class HeaderRecord {
00016
00017 public:
00018
00023     HeaderRecord(int numFields = 6);
00024
00025     // Getter methods
00030     std::string getFileStructureType() const { return fileStructureType; };
00035     std::string getVersion() const { return version; };
00036
00041     int getHeaderRecordSize() const { return headerRecordSize; };
00042
00047     int getRecordSizeInteger() const { return recordSizeInteger; };
00048
00053     std::string getSizeFormatType() const { return sizeFormatType; };
00054
00059     std::string getIndexFileName() const { return indexFileName; };
00060
00065     std::string getIndexFileSchemaInfo() const { return indexFileSchemaInfo; };
00066
00071     int getRecordCount() const { return recordCount; };
00072
00077     int getFieldsPerRecord() const { return fieldsPerRecord; };
00078
00084     std::string getFieldType(const std::string& fieldName) const;
00085
00090     std::string getPrimaryKey() const { return primaryKey; };
00091
00092
00093     // Setter methods for each private member
00098     void setFileStructureType(const std::string& value);
00103     void setVersion(const std::string& value);
00104
00109     void setHeaderRecordSize(int value);
00110
00115     void setRecordSizeInteger(int value);
00116

```

```

00121     void setSizeFormatType(const std::string& value);
00122
00127     void setIndexFileName(const std::string& value);
00128
00133     void setIndexFileSchemaInfo(const std::string& value);
00134
00139     void setRecordCount(int value);
00140
00145     void setFieldsPerRecord(int value);
00146
00153     void setField(int index, const std::string& fieldName, const std::string& fieldType);
00154
00159     void setPrimaryKey(const std::string& value);
00160
00165     std::string Pack();
00166
00171     void Unpack(std::string str);
00172
00176     void Print();
00177
00182     std::vector<std::string> getFieldOrder() const;
00183
00184 private:
00185     std::string fileStructureType;
00186     std::string version;
00187     int headerRecordSize;
00188     int recordSizeInteger;
00189     std::string sizeFormatType;
00190     std::string indexFileName;
00191     std::string indexFileSchemaInfo;
00192     int recordCount;
00193     int fieldsPerRecord;
00194     std::vector<std::pair<std::string, std::string>> fields;
00195     std::string primaryKey;
00196 };
00197
00198 #endif // HEADER_RECORD_H

```

4.15 Project4Code/primaryKeyIndex.cpp File Reference

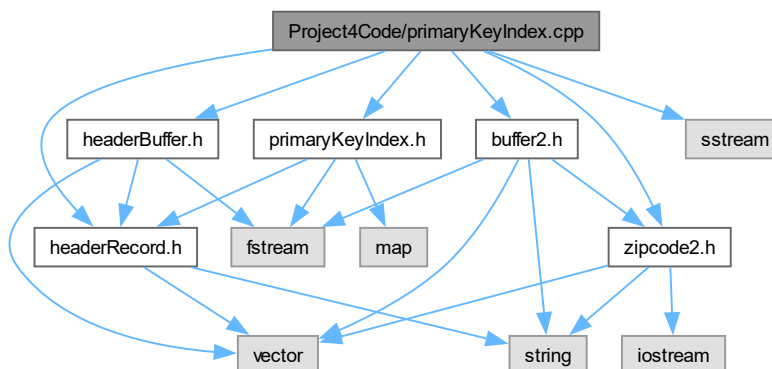
This file contains the implementation of the [PrimaryKeyIndex](#) class.

```

#include "primaryKeyIndex.h"
#include "zipcode2.h"
#include "headerBuffer.h"
#include "headerRecord.h"
#include "buffer2.h"
#include <sstream>

```

Include dependency graph for primaryKeyIndex.cpp:



4.15.1 Detailed Description

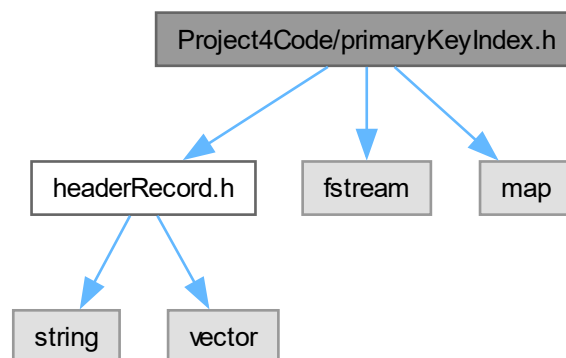
This file contains the implementation of the [PrimaryKeyIndex](#) class.

4.16 Project4Code/primaryKeyIndex.h File Reference

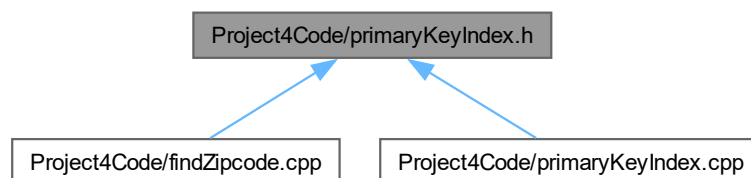
This file contains the declaration of the [PrimaryKeyIndex](#) class.

```
#include "headerRecord.h"  
#include <fstream>  
#include <map>
```

Include dependency graph for primaryKeyIndex.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [PrimaryKeyIndex](#)

An index based on the primary key of the zipcode records.

4.16.1 Detailed Description

This file contains the declaration of the [PrimaryKeyIndex](#) class.

4.17 primaryKeyIndex.h

[Go to the documentation of this file.](#)

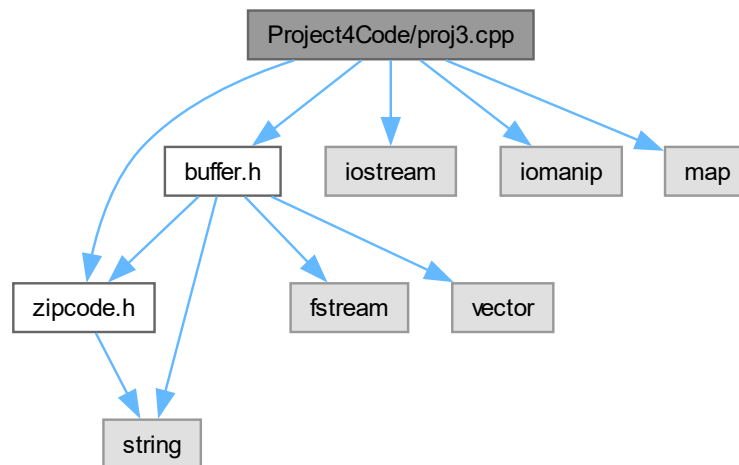
```
00001
00010 #ifndef PRIMARY_KEY_INDEX_H
00011 #define PRIMARY_KEY_INDEX_H
00012
00013 #include "headerRecord.h"
00014 #include <fstream>
00015 #include <map>
00016
00021 class PrimaryKeyIndex {
00022 public:
00026     PrimaryKeyIndex();
00027
00031     ~PrimaryKeyIndex();
00032
00038     bool createIndex(std::fstream& inFile);
00039
00044     void writeIndexToFile(std::fstream& outFile);
00045
00050     void readIndexFromFile(std::fstream& inFile);
00051
00057     int searchIndex(int zipcode);
00058
00064     void setHeaderRecord(HeaderRecord& dataHR, HeaderRecord& indexHR);
00065
00066 private:
00067     std::fstream indexFile;
00068     std::map<int, int> intIndex;
00069     int headerOffset = 450;
00070 };
00071
00072 #endif // PRIMARY_KEY_INDEX_H
```

4.18 Project4Code/proj3.cpp File Reference

A program to analyze geographical data from a CSV file and display extreme zip codes for each state.

```
#include "zipcode.h"
#include "buffer.h"
#include <iostream>
#include <iomanip>
#include <map>
```

Include dependency graph for proj3.cpp:



Functions

- void `processData` (`Buffer` &buffer, `std::map< std::string, double >` &easternmost, `std::map< std::string, double >` &westernmost, `std::map< std::string, double >` &northernmost, `std::map< std::string, double >` &southernmost)
< Include the map library for storing the extreme coordinates
- void `printData` (const `std::map< std::string, double >` &easternmost, const `std::map< std::string, double >` &westernmost, const `std::map< std::string, double >` &northernmost, const `std::map< std::string, double >` &southernmost)
Function to print the extreme coordinates for each state.
- int `main` ()
The main function of the program.

4.18.1 Detailed Description

A program to analyze geographical data from a CSV file and display extreme zip codes for each state.

This program reads geographical data from a CSV file, processes it to find extreme zip codes (easternmost, westernmost, northernmost, and southernmost) for each state, and displays the results in a formatted table. It utilizes the `RecordBuffer` class to store and retrieve records, and various helper functions for data processing and output formatting.

Author

Daniel Leone @starID [REDACTED]

Date

03/09/2024

4.18.2 Function Documentation

4.18.2.1 main()

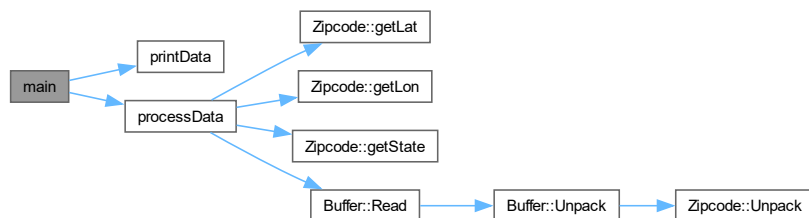
```
int main ( )
```

The main function of the program.

Returns

0 if the program executed successfully.

Here is the call graph for this function:



4.18.2.2 printData()

```
void printData (
    const std::map< std::string, double > & easternmost,
    const std::map< std::string, double > & westernmost,
    const std::map< std::string, double > & northernmost,
    const std::map< std::string, double > & southernmost )
```

Function to print the extreme coordinates for each state.

Parameters

<i>easternmost, westernmost, northernmost, southernmost</i>	Maps holding the extreme coordinates for each state.
---	--

Here is the caller graph for this function:



4.18.2.3 processData()

```
void processData (
    Buffer & buffer,
    std::map< std::string, double > & easternmost,
    std::map< std::string, double > & westernmost,
    std::map< std::string, double > & northernmost,
    std::map< std::string, double > & southernmost )
```

< Include the map library for storing the extreme coordinates

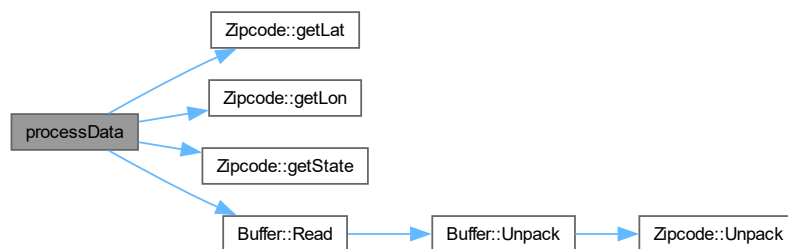
Function to read and process the data from the file.

< Include the [Zipcode](#) class < Include the [Buffer](#) class Include the iostream library for input/output operations < Include the iomanip library for output formatting

Parameters

<i>buffer</i>	The Buffer object.
<i>easternmost,westernmost,northernmost,southernmost</i>	Maps to hold the extreme coordinates for each state.

Here is the call graph for this function:



Here is the caller graph for this function:

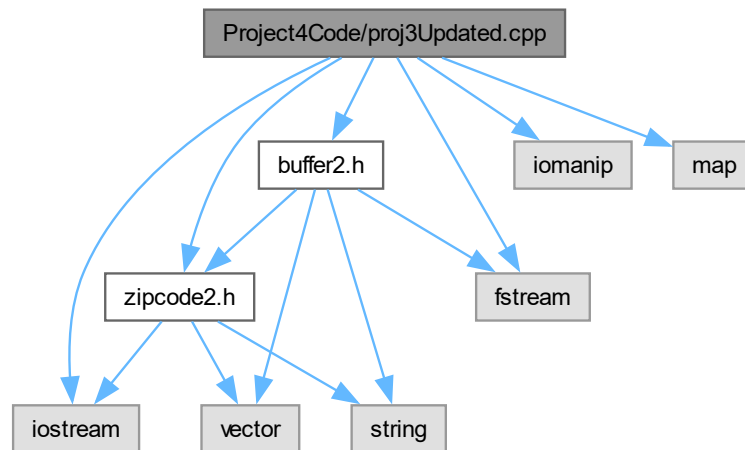


4.19 Project4Code/proj3Updated.cpp File Reference

A program to analyze geographical data from a ZCR file and display extreme zip codes for each state.

```
#include "zipcode2.h"
#include "buffer2.h"
#include <fstream>
#include <iostream>
#include <iomanip>
#include <map>
```

Include dependency graph for proj3Updated.cpp:



Functions

- void [processData](#) ([Buffer2](#) &buffer, std::fstream &file, std::map< std::string, double > &easternmost, std::map< std::string, double > &westernmost, std::map< std::string, double > &northernmost, std::map< std::string, double > &southernmost)
 - < Include the map library for storing the extreme coordinates
- void [printData](#) (const std::map< std::string, double > &easternmost, const std::map< std::string, double > &westernmost, const std::map< std::string, double > &northernmost, const std::map< std::string, double > &southernmost)
 - Function to print the extreme coordinates for each state.
- int [main](#) ()
 - The main function of the program.

4.19.1 Detailed Description

A program to analyze geographical data from a ZCR file and display extreme zip codes for each state.

This program reads geographical data from a ZCR file, processes it to find extreme zip codes (easternmost, westernmost, northernmost, and southernmost) for each state, and displays the results in a formatted table. It utilizes the RecordBuffer class to store and retrieve records, and various helper functions for data processing and output formatting.

Author

Daniel Leone @starID [REDACTED]

Date

04/05/2024

4.19.2 Function Documentation

4.19.2.1 main()

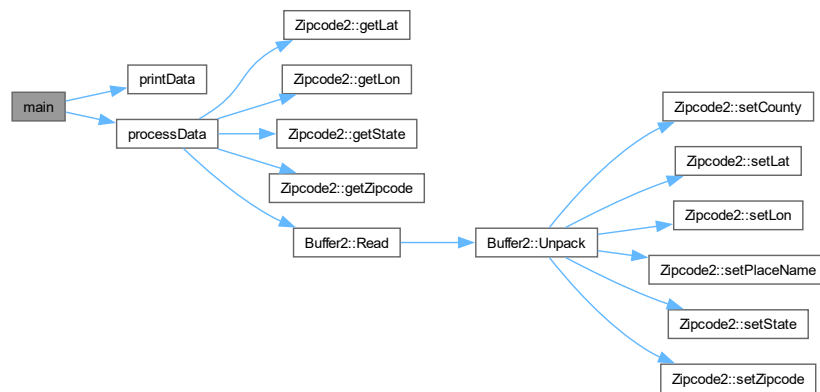
```
int main ( )
```

The main function of the program.

Returns

0 if the program executed successfully.

Here is the call graph for this function:



4.19.2.2 printData()

```
void printData (
    const std::map< std::string, double > & easternmost,
    const std::map< std::string, double > & westernmost,
    const std::map< std::string, double > & northernmost,
    const std::map< std::string, double > & southernmost )
```

Function to print the extreme coordinates for each state.

Parameters

<i>easternmost, westernmost, northernmost, southernmost</i>	Maps holding the extreme coordinates for each state.
---	--

Here is the caller graph for this function:



4.19.2.3 processData()

```

void processData (
    Buffer2 & buffer,
    std::fstream & file,
    std::map< std::string, double > & easternmost,
    std::map< std::string, double > & westernmost,
    std::map< std::string, double > & northernmost,
    std::map< std::string, double > & southernmost )
  
```

< Include the map library for storing the extreme coordinates

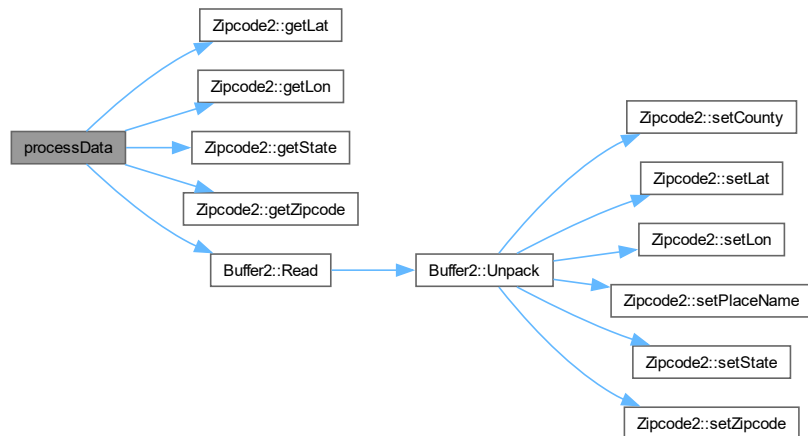
Function to read and process the data from the file.

< Include the [Zipcode2](#) class Include the [Buffer2](#) class Include the fstream library for file input/output operations
Include the iostream library for input/output operations Include the iomanip library for output formatting

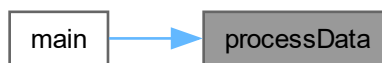
Parameters

<i>buffer</i>	The Buffer2 object.
<i>easternmost,westernmost,northernmost,southernmost</i>	Maps to hold the extreme coordinates for each state.

Here is the call graph for this function:



Here is the caller graph for this function:

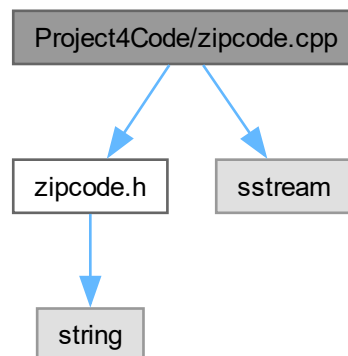


4.20 Project4Code/zipcode.cpp File Reference

This file contains the implementation of the [Zipcode](#) class. It is used for project 3.

```
#include "zipcode.h"
#include <sstream>
```

Include dependency graph for zipcode.cpp:



4.20.1 Detailed Description

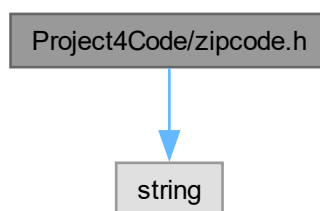
This file contains the implementation of the [Zipcode](#) class. It is used for project 3.

4.21 Project4Code/zipcode.h File Reference

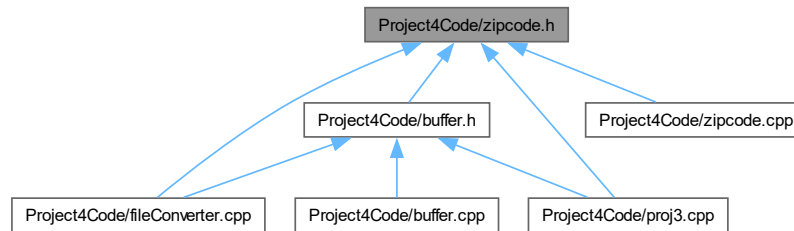
This file contains the declaration of the [Zipcode](#) class. It is used for project 4.

```
#include <string>
```

Include dependency graph for zipcode.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Zipcode](#)

A class for handling zipcode records with associated geographic information.

4.21.1 Detailed Description

This file contains the declaration of the [Zipcode](#) class. It is used for project 4.

4.22 zipcode.h

[Go to the documentation of this file.](#)

```

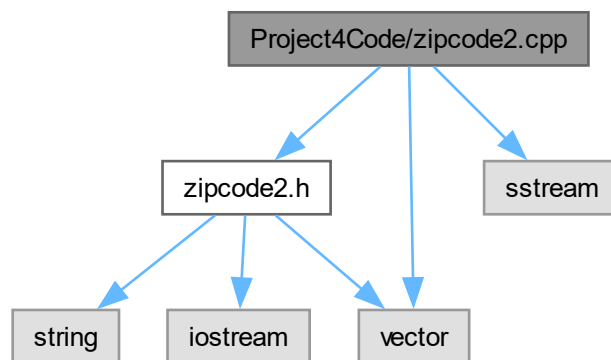
00001
00010 #ifndef ZIPCODE_H
00011 #define ZIPCODE_H
00012
00013 #include <string>
00014
00015 class Zipcode {
00016 public:
00020     Zipcode();
00021
00026     std::string Pack();
00027
00032     void Unpack(std::string str);
00033
00038     int getZipcode() { return zipcode; };
00039
00044     std::string getPlaceName() { return placeName; };
00045
00050     std::string getState() { return state; };
00051
00056     std::string getCounty() { return county; };
00057
00062     double getLat() { return lat; };
00063
00068     double getLon() { return lon; };
00069
00070
00075     void setZipcode(int zipcode);
00076
00081     void setPlaceName(std::string placeName);
00082
00087     void setState(std::string state);
00088
00093     void setCounty(std::string county);
00094
00099     void setLat(double lat);
00100
00105     void setLon(double lon);
  
```

```
00106
00107 private:
00108     int zipcode;
00109     std::string placeName;
00110     std::string state;
00111     std::string county;
00112     double lat;
00113     double lon;
00114 };
00115
00116 #endif
```

4.23 Project4Code/zipcode2.cpp File Reference

This file contains the implementation of the [Zipcode2](#) class. It is used for project 4.

```
#include "zipcode2.h"
#include <sstream>
#include <vector>
Include dependency graph for zipcode2.cpp:
```



4.23.1 Detailed Description

This file contains the implementation of the [Zipcode2](#) class. It is used for project 4.

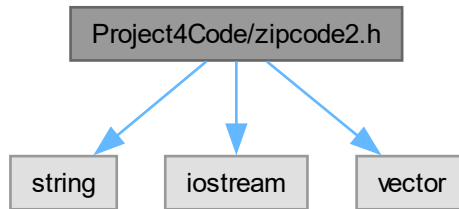
4.24 Project4Code/zipcode2.h File Reference

This file contains the declaration of the [Zipcode2](#) class. It is used for project 4.

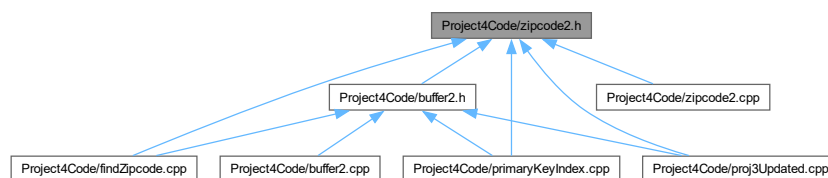
```
#include <string>
#include <iostream>
```

```
#include <vector>
```

Include dependency graph for zipcode2.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Zipcode2](#)

A class for handling zipcode records with associated geographic information.

4.24.1 Detailed Description

This file contains the declaration of the [Zipcode2](#) class. It is used for project 4.

4.25 zipcode2.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef ZIPCODE2_H
00011 #define ZIPCODE2_H
00012
00013 #include <string>
00014 #include <iostream>
00015 #include <vector>
00016
00017 class Zipcode2 {
00018 public:
00022     Zipcode2();
00023
00028     int getZipcode() { return zipcode; };
  
```

```
00029
00034     std::string getLocation() { return placeName; };
00035
00040     std::string getState() { return state; };
00041
00046     std::string getCounty() { return county; };
00047
00052     double getLat() { return lat; };
00053
00058     double getLon() { return lon; };
00059
00060
00065     void setZipcode(int zipcode);
00066
00071     void setPlaceName(std::string placeName);
00072
00077     void setState(std::string state);
00078
00083     void setCounty(std::string county);
00084
00089     void setLat(double lat);
00090
00095     void setLon(double lon);
00096
00102     std::string Pack(const std::vector<std::string>& fieldOrder);
00103
00108     void Unpack(std::string str);
00109
00113     void Print();
00114
00119     bool isEmpty() const;
00120
00121 private:
00122     int zipcode;
00123     std::string placeName;
00124     std::string state;
00125     std::string county;
00126     double lat;
00127     double lon;
00128 };
00129
00130 #endif
```